

VULNERABILITIES OF ARCHITECTURES, DESIGNS, & SOLUTION ELEMENTS

Objectives

- Explore how to assess and mitigate the vulnerabilities of client-based vs. server-based systems
- Examine how to assess and mitigate the vulnerabilities of database, cryptographic, industrial control, and embedded systems
- Learn about assessing and mitigating vulnerabilities of virtualized, cloud-based, distributed, and IoT systems
- Learn how to assess and mitigate vulnerabilities of containerization, microservices, and serverless technologies
- Explore how to assess and mitigate the vulnerabilities of high-performance, edge computing, and trusted execution environments



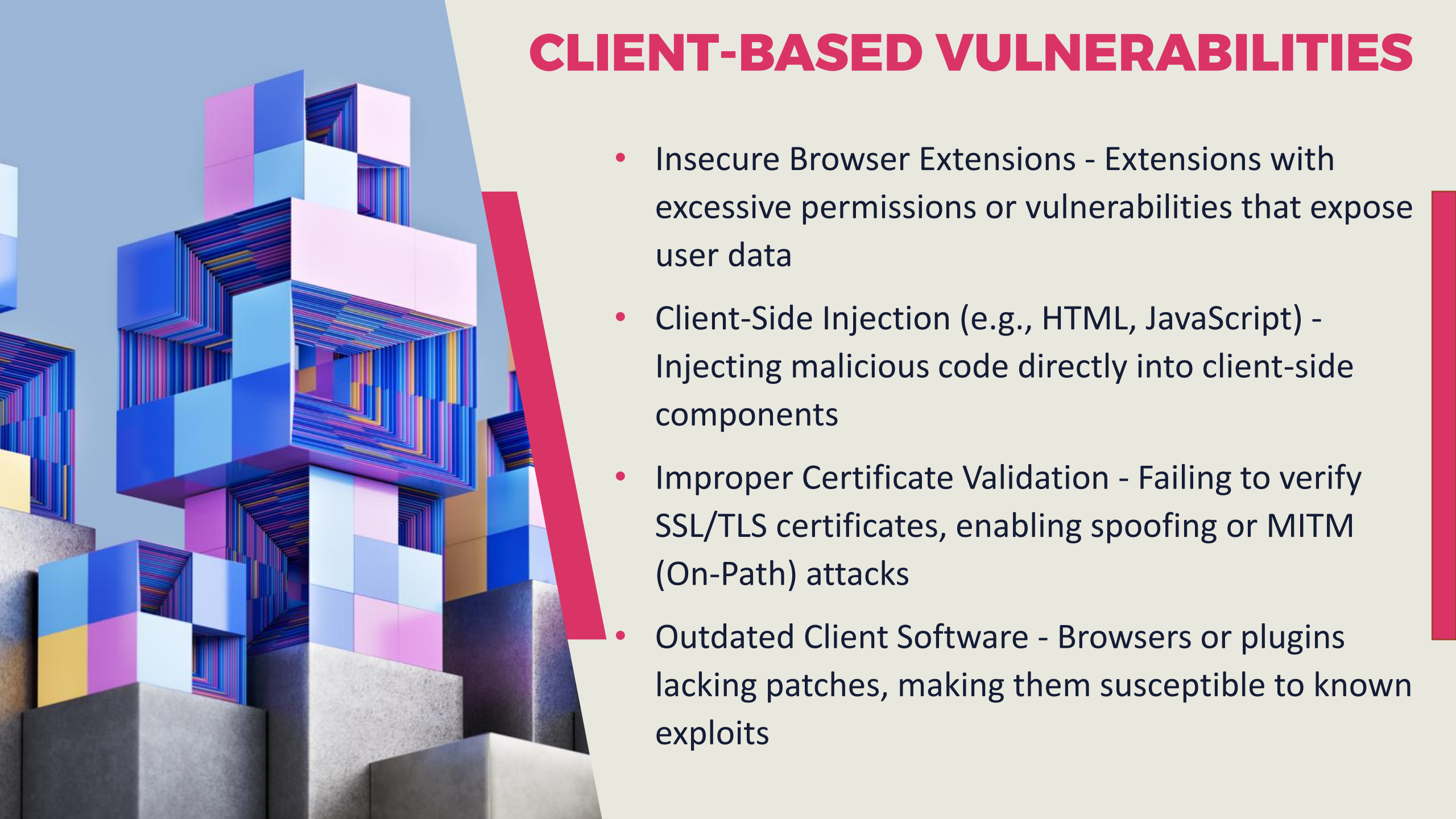
CLIENT-BASED VULNERABILITIES

- Cross-Site Scripting (XSS) - Malicious scripts injected into trusted websites, executed in the user's browser
- Clickjacking - UI deception that tricks users into clicking hidden or disguised elements
- Man-in-the-Browser (MitB) - Malware that intercepts and manipulates browser transactions in real time
- Drive-by Downloads - Automatic download of malicious software when visiting compromised or malicious sites

CLIENT-BASED VULNERABILITIES

- Session Hijacking - Exploiting active sessions to impersonate users and gain unauthorized access
- Local Storage Exploits - Abusing browser storage (e.g., cookies, localStorage) to extract sensitive data
- DOM-Based Attacks - Manipulating the Document Object Model (DOM) to execute malicious actions client-side
- Unvalidated Redirects and Forwards - Redirecting users to malicious destinations without proper validation





CLIENT-BASED VULNERABILITIES

- Insecure Browser Extensions - Extensions with excessive permissions or vulnerabilities that expose user data
- Client-Side Injection (e.g., HTML, JavaScript) - Injecting malicious code directly into client-side components
- Improper Certificate Validation - Failing to verify SSL/TLS certificates, enabling spoofing or MITM (On-Path) attacks
- Outdated Client Software - Browsers or plugins lacking patches, making them susceptible to known exploits

MITIGATING CLIENT-SIDE VULNERABILITIES



Subresource Integrity (SRI)

Enables browsers to verify that resources fetched (from a CDN) are supplied without unforeseen manipulation



Content Security Policy

Prevents XSS, clickjacking, and other code injection



Behavioral detection

Tools such as ESLint, Esprima, and Jalangi2



JavaScript analyzers

Prevents XSS, clickjacking, and other code injection



Next-generation Endpoint Detection and Response (XDR)

SERVER-SIDE VULNERABILITIES

- SQL injection attacks
 - Attackers exploit web application code to get access to the backend database by injecting SQL commands into the uniform resource identifier (URI)
- Server-side request forgery (SSRF)
 - Allows an attacker to induce the server-side application to make requests to an unintended location
 - This can give the threat actor access to internal systems, sensitive information, or other threat vectors
- Remote Code Execution (RCE)
 - Attackers exploit flaws to execute arbitrary code on the server, often leading to full system compromise





SERVER-SIDE VULNERABILITIES

- Authentication Bypass
 - Weak or misconfigured authentication mechanisms allow attackers to gain unauthorized access
- Insecure Deserialization
 - Malicious objects are injected into serialized data streams, leading to code execution or privilege escalation
- Directory Traversal
 - Attackers manipulate file paths to access restricted directories and files on the server

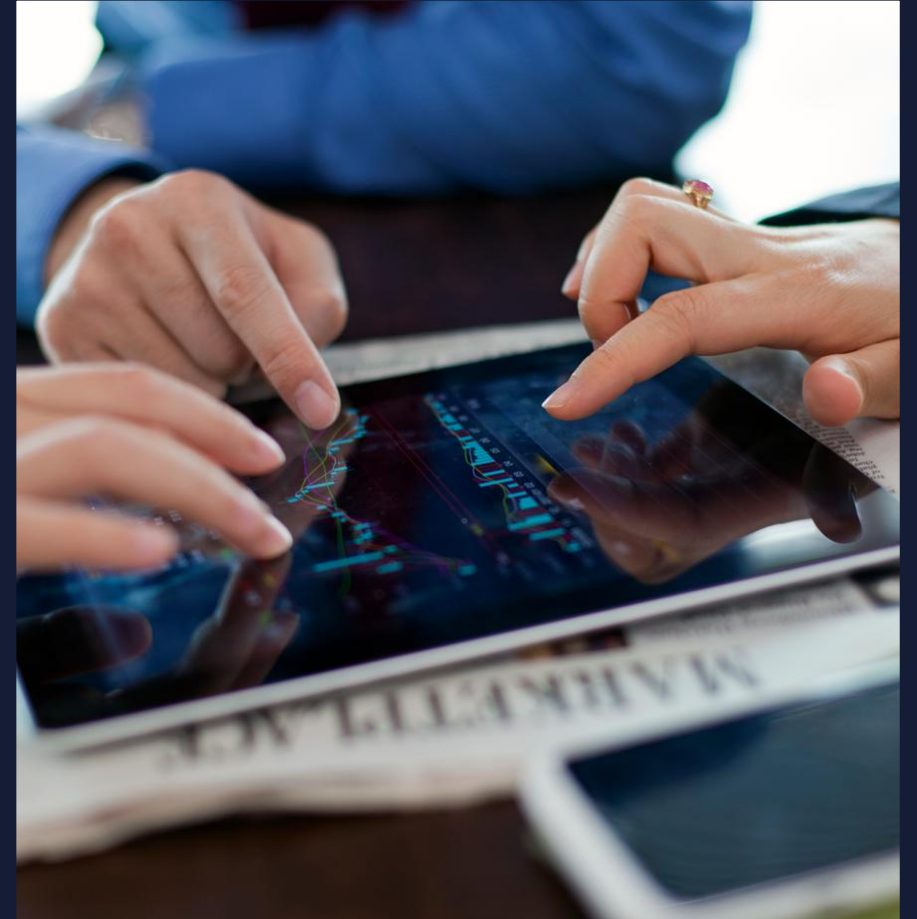
SERVER-SIDE VULNERABILITIES

- Privilege Escalation
 - Exploiting flaws to gain higher-level access than intended, often through misconfigured services or kernel vulnerabilities
- Improper Error Handling
 - Detailed error messages reveal sensitive system information, aiding reconnaissance and exploitation



MITIGATING SERVER-SIDE VULNERABILITIES

- Parameterized Queries & ORM Use
 - Prevent SQL injection by avoiding dynamic query construction
- -Input Validation & Whitelisting
 - Validate all incoming data, especially from APIs and user forms
- Secure Configuration & Patch Management
 - Harden servers, disable unused services, and apply patches promptly
- Access Control & Least Privilege
 - Enforce strict access policies and minimize privilege levels for services and users



MODERN DATABASE TYPES

Relational
(RDBMS like
MySQL and
Postgres)

NoSQL (Non-
relational) like
AWS DynamoDB
or GCP Datastore)

**Document/key-
value** (memory-
only like Redis)

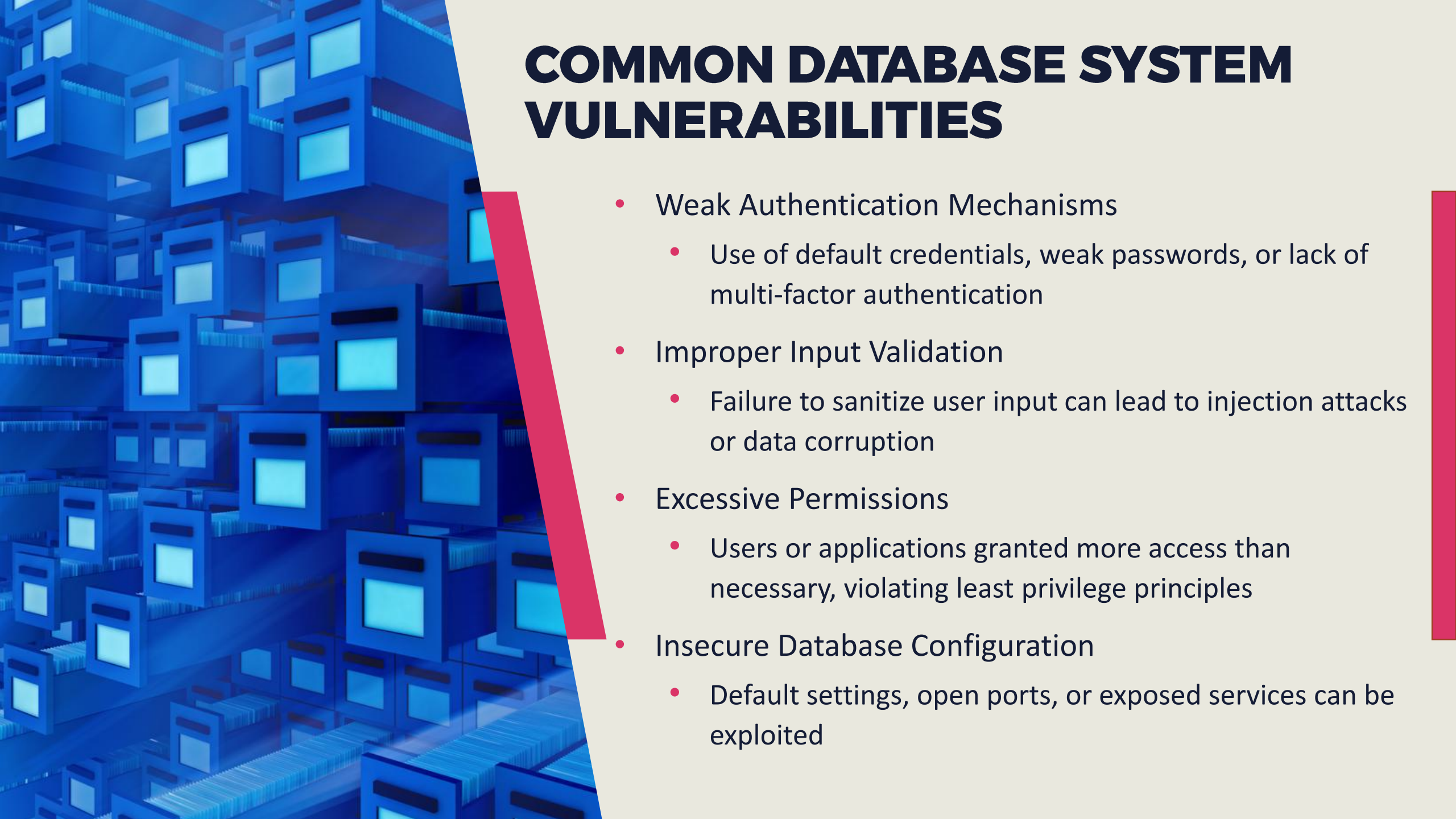
Graph (AWS
Neptune)

Blockchain

COMMON DATABASE SYSTEM VULNERABILITIES

- SQL Injection
 - Malicious input manipulates SQL queries to access or modify data without authorization
- Buffer Overflows
 - Improper handling of memory allocation can allow attackers to overwrite memory and execute arbitrary code
- Privilege Escalation
 - Exploiting misconfigured roles or flaws to gain higher-level access than intended
- Unpatched Database Software
 - Outdated versions may contain known vulnerabilities that can be exploited





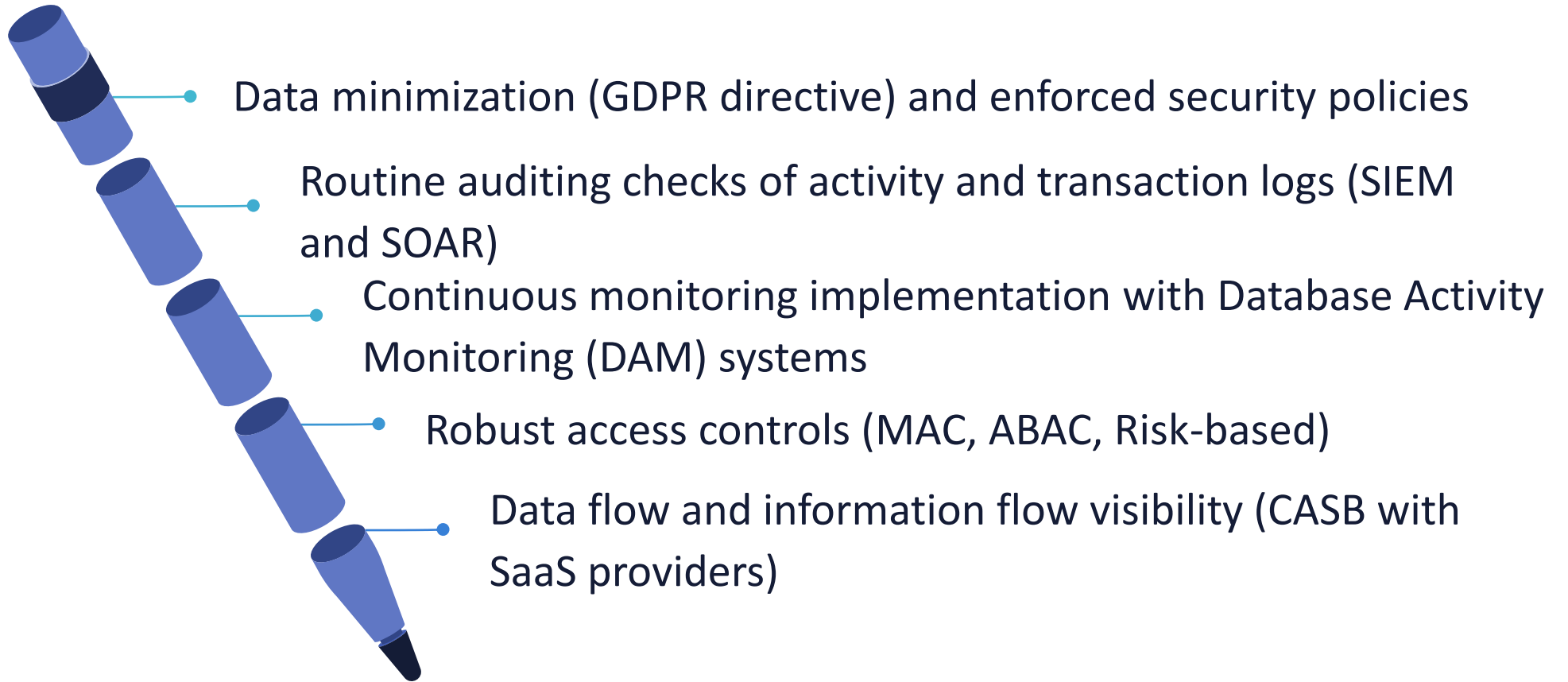
COMMON DATABASE SYSTEM VULNERABILITIES

- Weak Authentication Mechanisms
 - Use of default credentials, weak passwords, or lack of multi-factor authentication
- Improper Input Validation
 - Failure to sanitize user input can lead to injection attacks or data corruption
- Excessive Permissions
 - Users or applications granted more access than necessary, violating least privilege principles
- Insecure Database Configuration
 - Default settings, open ports, or exposed services can be exploited

COMMON DATABASE SYSTEM VULNERABILITIES

- Data Leakage via Error Messages
 - Verbose error messages may reveal schema details or sensitive system information
- Insecure APIs or Interfaces
 - Poorly secured database endpoints can be targeted for unauthorized access or data exfiltration
- Lack of Encryption (Data at Rest & in Transit)
 - Unencrypted data is vulnerable to interception or theft
- Improper Session Management
 - Sessions not timed out or invalidated properly can be hijacked

DATABASE SECURITY COUNTERMEASURES



MAIN CATEGORIES OF CRYPTANALYSIS



1. Brute Force Attack

Aspect	Description
Method	Systematically tries every possible key or password until the correct one is found.
Target	The cryptographic key space.
Assumption	The algorithm is secure, but the key is discoverable through exhaustive search.
Mitigation	Use long, complex keys; enforce rate limiting; implement account lockouts.
Example	Trying all 2^{64} keys in a symmetric cipher until the correct one decrypts the message.

MAIN CATEGORIES OF CRYPTANALYSIS



2. Implementation Attack

Aspect	Description
Method	Exploits flaws in how the cryptographic algorithm is implemented—not the algorithm itself.
Target	Software bugs, insecure libraries, poor coding practices.
Assumption	The algorithm is theoretically secure, but real-world implementation introduces vulnerabilities.
Mitigation	Use vetted libraries, conduct code reviews, apply secure coding standards.
Example	Exploiting a buffer overflow in a poorly coded encryption routine to execute arbitrary code.

MAIN CATEGORIES OF CRYPTANALYSIS



3. Side-Channel Attack

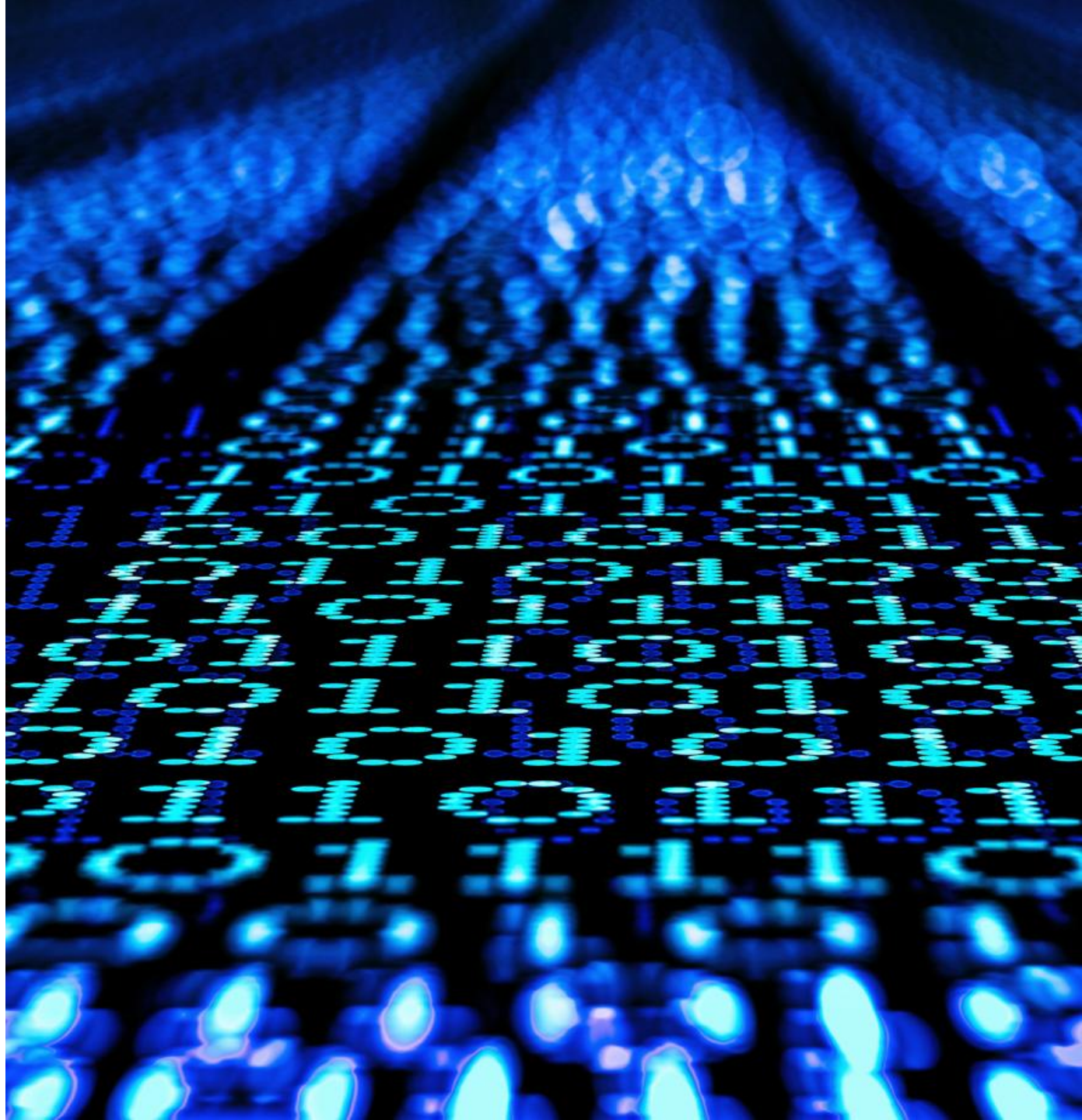
Aspect	Description
Method	Gathers information from physical or observable characteristics of the system (e.g., timing, power usage, EM emissions).
Target	Cryptographic operations as they run on hardware.
Assumption	The attacker has physical or observational access to the system.
Mitigation	Use constant-time algorithms, shield hardware, randomize execution paths.
Example	Measuring power fluctuations to deduce bits of a private key during decryption.

MODERN CRYPTOGRAPHIC VULNERABILITIES



COUNTERMEASURES FOR CRYPTOGRAPHIC ATTACKS

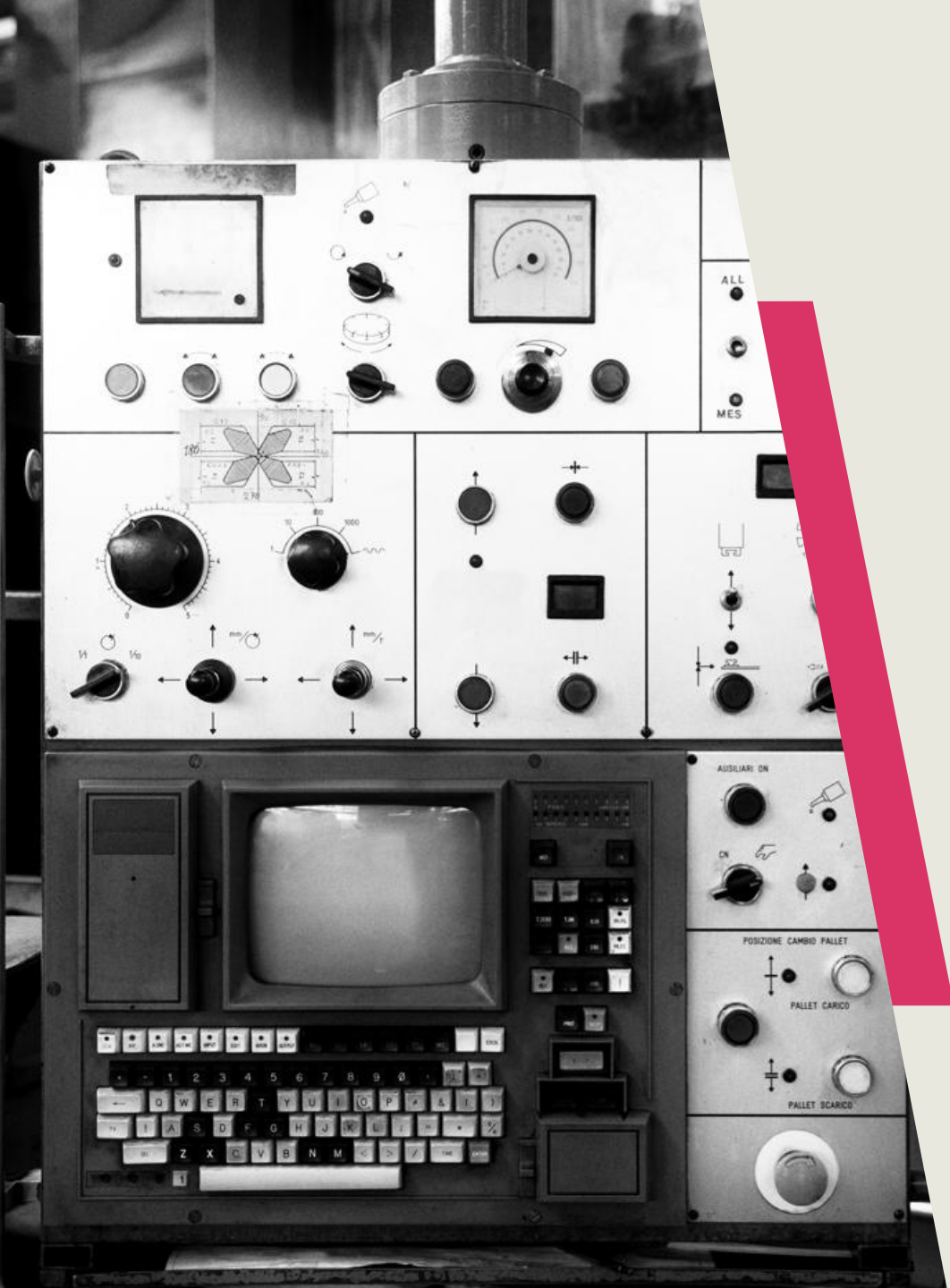
- Ensuring that the cryptographic system is implemented properly
- Updating and testing cryptographic algorithms and protocols (not deprecated)
- Never embedding cryptographic material in repositories, databases, or in API calls
- Using trusted execution environments such as Trusted Platform Module (TPM), hardware security module (HSM), and secure enclaves



COUNTERMEASURES FOR CRYPTOGRAPHIC ATTACKS

- Tested encryption processes on data-at-rest, data-in-transit, and data-in-use
- Usage of ephemeral and temporary session keys whenever possible (forward secrecy)
- Staying up-to-date on new malware, variants, and zero-day threats



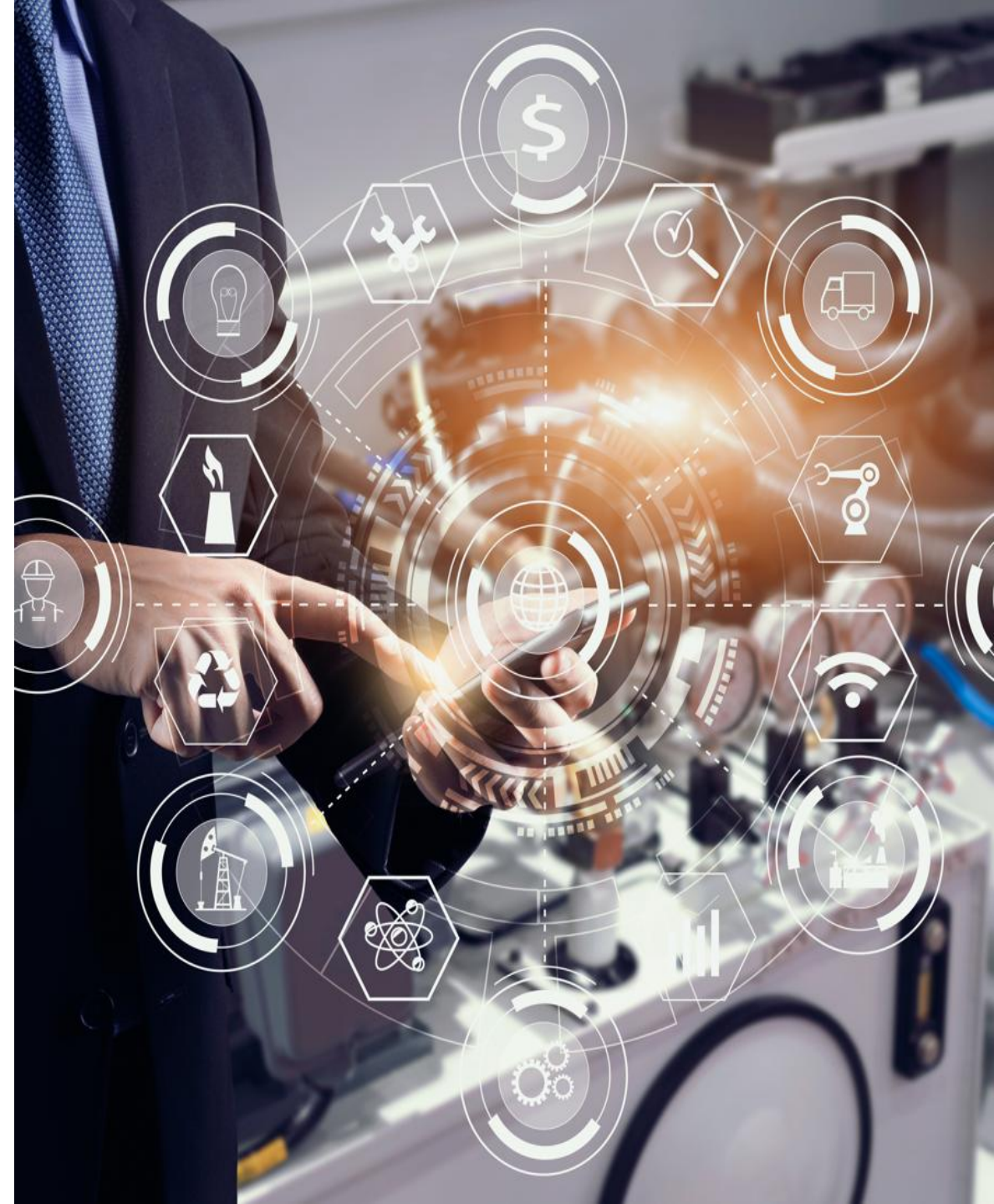


INDUSTRIAL CONTROL (ICS) AND EMBEDDED SYSTEMS

- Devices in ICS and embedded environments make a striking target for attackers due to their common absence of strong security controls and insufficient testing for vulnerabilities
- Threat agents have increased attacks targeting these components, particularly in areas like food and agriculture, chemical, water treatment, and manufacturing

INDUSTRIAL CONTROL AND EMBEDDED SYSTEMS

- Many of the new vulnerabilities to embedded devices allow for:
 - Remote exploitation
 - Execution with low attack complexity
 - Taking control of affected systems
 - Manipulating and modifying settings
 - Escalating privileges
 - Bypassing security controls
 - Stealing data
 - Crashing systems



The background image shows a person's hands holding a smartphone in front of an industrial machine. Overlaid on the image is a grid of hexagonal icons. Some icons are white on a dark background, while others are dark on a light background. The icons represent various industrial and technological concepts: a factory, a gear, a Wi-Fi signal, a person's head, a recycling symbol, a group of people, a flag, a speech bubble, a truck, and a circuit board. The overall theme is industrial control and embedded systems.

INDUSTRIAL CONTROL (ICS) AND EMBEDDED SYSTEMS

- Real-world use cases
 - Unpatched exploits in Siemens' Ruggedcom APE1808, an industry-grade application processing engine (APE) module
 - Siemens' Scalance W-700 devices, an industry-grade suite of networking and bus systems, have been widely targeted lately

COUNTERMEASURES FOR ICS ATTACKS

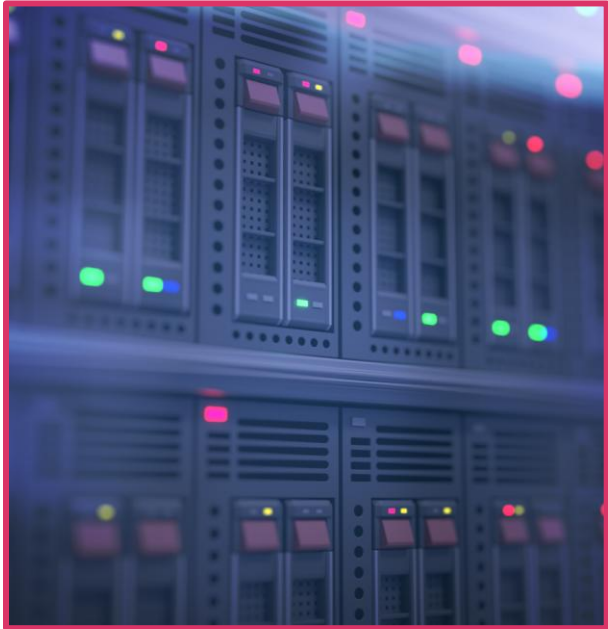


- The **EMB3D** threat model magnifies several prevailing models, including common weakness enumeration (CWE), MITRE ATT&CK, and common vulnerabilities and exposures (CVE)
 - It has a precise focus on embedded devices and fosters a knowledge base of cyber threats to devices
 - Advantages include observing "in the field" environments
 - It uses proof-of-concept and/or theoretic research

VIRTUALIZATION RISKS



- Virtual machine (VM) sprawl and lack of management
- VM hopping and escape
- Sensitive data in a VM
 - Passwords, personal data, bash profiles, bash history files, encryption keys, license keys, data, and image captures
- Lack of visibility into and control over virtual networks
 - Traffic over virtual networks may not be visible to physical network security protection devices, such as network-based intrusion detection system (NIDS)/network-based intrusion prevention system (NIPS)



VIRTUALIZATION RISKS

- Security of offline and dormant VM
 - Security of pre-configured (Golden Image) VM/active VMs
- Resource exhaustion
 - Resource-intensive software can consume resources in a physical server when it is deployed in multiple VMs. For instance, antivirus and other security software interrupt every call to disk or memory to monitor and prevent security incidents such as cracking or viruses
- Unauthorized access to hypervisor
 - Hyperjacking

HYPERJACKING



- An attack that is targeted at exploiting a system from the outside to disrupt or inject a rogue module into the system
- The attacker can control the virtual machines running on top of the same host and monitor their network traffic
- Normal security controls are futile since the operating system will not be aware that the machine has been compromised
- Hyperjacking countermeasures
 - Hypervisor security management must be kept separate from regular traffic
 - Visibility into privileged insiders (honey tokens)
 - Tested patch management

VM SPRAWL COUNTERMEASURES

- Centralized VM Management Tools
 - Use hypervisor-native or third-party platforms (e.g., vCenter, SCVMM) to monitor, control, and audit VM creation and lifecycle
- Strict Provisioning Policies
 - Enforce role-based access control (RBAC) and approval workflows for creating new VMs
 - No ad hoc deployments
- Automated Inventory & Discovery
 - Continuously scan and catalog all active VMs to detect rogue or orphaned instances



VM SPRAWL

COUNTERMEASURES

- Lifecycle Governance
 - Define clear policies for VM retirement, patching, and resource allocation
 - Use tagging and metadata to track ownership and purpose
- Resource Quotas & Limits
 - Prevent over-provisioning by setting quotas per user, team, or project
- Configuration Baselines
 - Apply hardened templates and secure defaults to all VM deployments
 - Align with CIS benchmarks or NIST guidelines



VM SPRAWL

COUNTERMEASURES

- Regular Auditing & Reporting
 - Schedule audits to identify unused, misconfigured, or non-compliant VMs. Integrate with SIEM tools for visibility
- Change Management Integration
 - Tie VM creation and modification to formal change control processes to ensure accountability and traceability
- Patch & Vulnerability Management
 - Ensure all VMs are included in automated patching cycles and vulnerability scans—no exceptions
- Decommissioning Protocols
 - Securely retire VMs with proper data sanitization, logging, and resource reclamation





THE CSA TREACHEROUS 12

1. Data breaches
2. Weak identity, credential, and access management
3. Insecure APIs
4. System and application vulnerabilities
5. Account hijacking
6. Malicious insiders

THE CSA TREACHEROUS 12

1. Advanced persistent threats (APTs)
2. Data loss
3. Insufficient due diligence
4. Abuse and nefarious use of cloud services
5. Denial of service
6. Shared technology issues

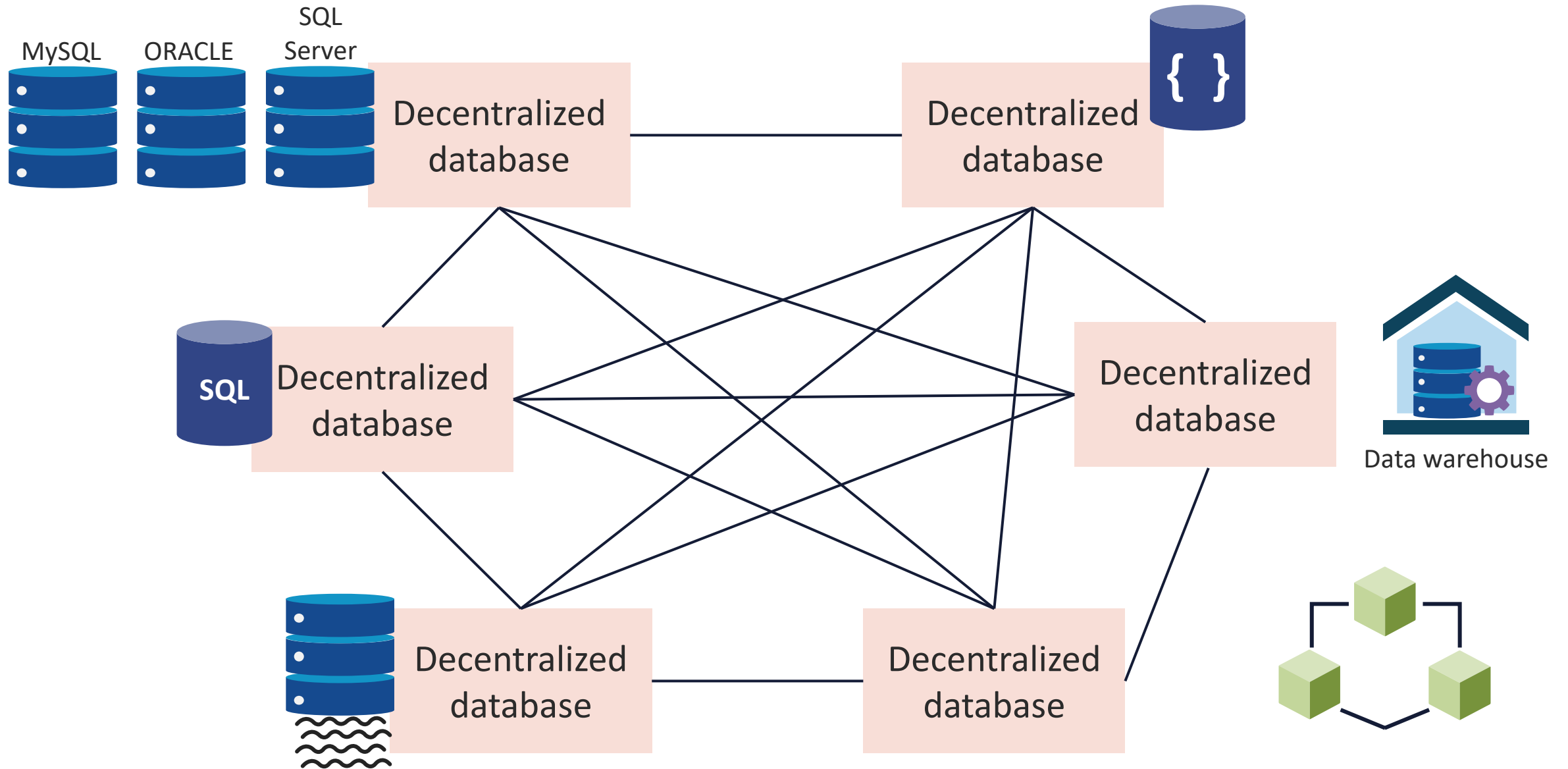




DISTRIBUTED SYSTEMS

- Unique attributes of distributed systems are:
 - No centralized servers to concentrate the countermeasures
 - Resource sharing means that data often resides in multiple locations accessed by more than one user, app, or system
 - Concurrency leads to processes and tasks running simultaneously yet not in exact timing
 - Scalability is an enabler for distributed denial-of-service (DoS) attacks
 - Fault tolerance is a benefit for resilience but adds additional targets

DISTRIBUTED DECENTRALIZED DATABASES



A hand is shown reaching out and touching a glowing point on a complex digital network. The network consists of numerous nodes, represented by laptop icons, connected by a web of white lines. The background is dark blue with circuit-like patterns. A bright yellow light emanates from the point of contact. A thick pink diagonal bar is positioned behind the hand and the network diagram.

DISTRIBUTED SYSTEM WEAKNESSES

- A distributed system needs more security measures than centralized systems, as there are many users, differentiated data, multiple sites, and distributed control
- Security engineers must consider many permutations of failures and errors that can happen at any time, independently or in combination with other error conditions



DISTRIBUTED SYSTEM WEAKNESSES

- In distributed communication systems, there are several types of exploits:
 - Passive eavesdroppers that monitor messages and collect private information – information leakage
 - Active attackers that not only eavesdrop but further corrupt messages by inserting new data or modifying existing data
 - Distributed denial-of-service (DDoS) and botnets
 - Unauthorized access through poor access controls

SECURING DISTRIBUTED SYSTEMS

- Strong Authentication & Access Control
 - Implement multi-factor authentication (MFA) across all nodes
 - Use role-based access control (RBAC) and least privilege principles
 - Centralize identity management with federated or single sign-on (SSO) systems
- End-to-End Encryption
 - Encrypt data in transit using TLS and data at rest using AES or equivalent with key management (Cloud KMS)
- Consistent Configuration Management
 - Use automated configuration tools (e.g., Ansible, Puppet) to enforce secure baselines
 - Apply secure defaults and disable unnecessary services across all nodes



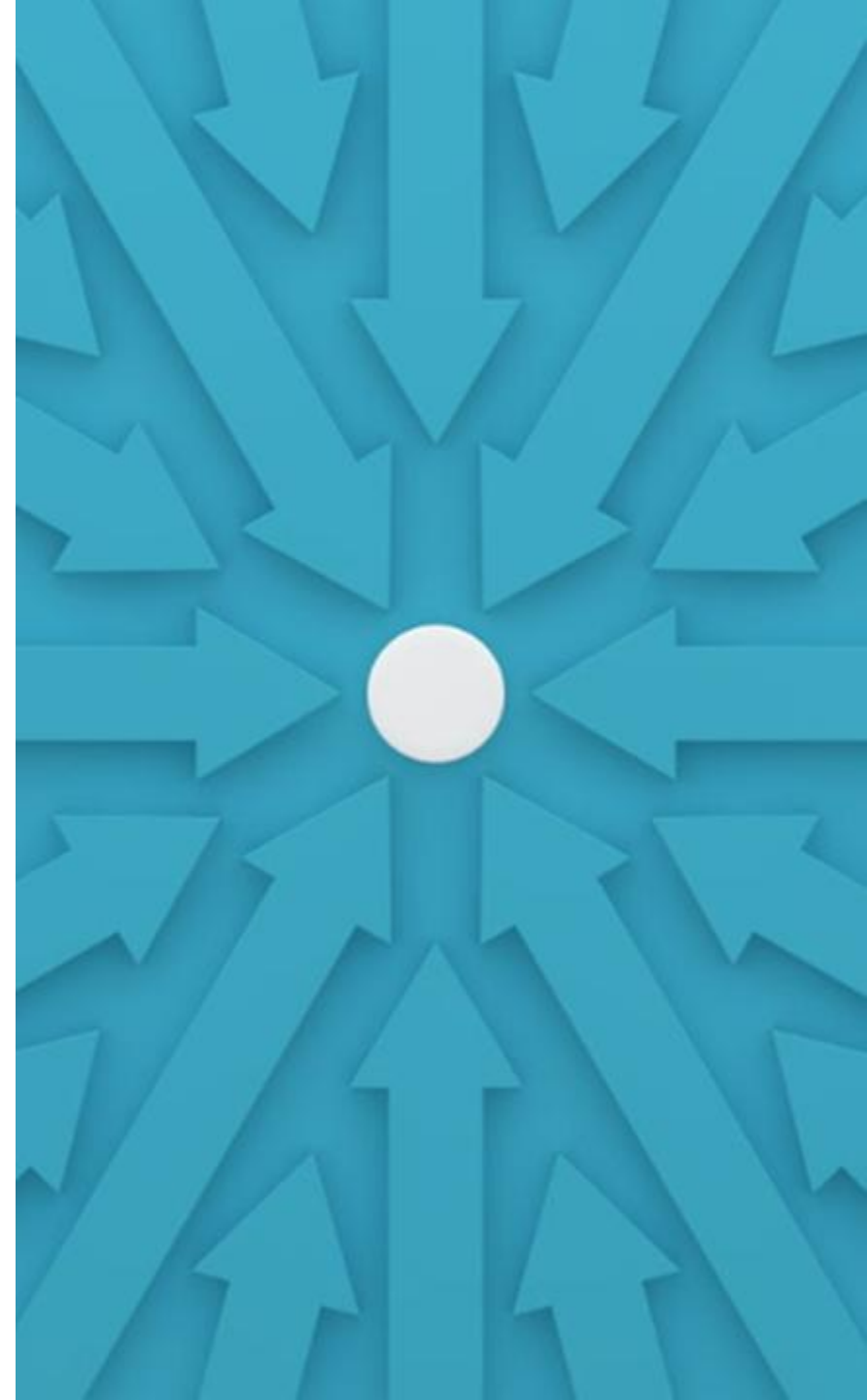


SECURING DISTRIBUTED SYSTEMS

- Patch Management & Vulnerability Scanning
 - Regularly scan all components for vulnerabilities using tools like Nessus or Qualys
- Centralized Logging & Monitoring
 - Aggregate logs from all nodes into a SIEM system
 - Monitor for anomalies, failed authentications, and unusual traffic patterns
- Secure APIs & Interfaces
 - Harden APIs with rate limiting, input validation, and authentication tokens
 - Use API gateways to enforce policy and monitor usage

SECURING DISTRIBUTED SYSTEMS

- Segmentation & Network Controls
 - Isolate critical systems using network segmentation and firewalls
 - Deploy intrusion detection/prevention systems (IDS/IPS) to monitor traffic between nodes
- Resilience & Fault Tolerance
 - Design for redundancy, failover, and graceful degradation
 - Use load balancing and distributed backups to ensure availability and integrity



ASSESSING INTERNET OF THINGS (IOT) VULNERABILITIES

In this demo...

- We will determine how to assess and mitigate the vulnerabilities of Internet of Things

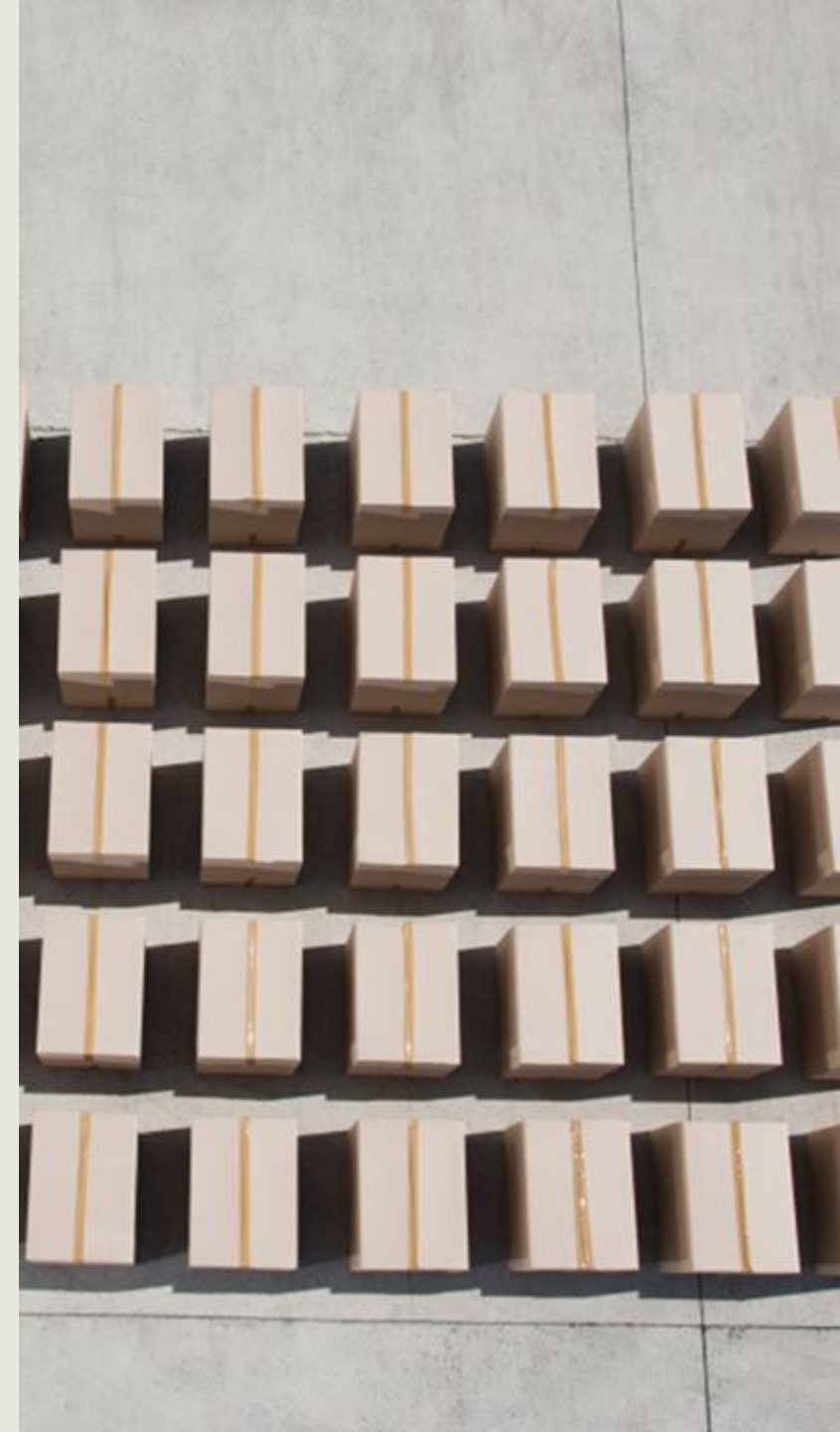


CONTAINERS

- A container is a discrete environment within an operating system (or a serverless architecture) where one or more applications can run and that is typically assigned all the resources and dependencies needed to function
- It is a modular and portable environment that includes the application binaries, software dependencies, and hardware requirements wrapped up into an independent, self-contained unit

MICROSERVICES

- Microservices are specific service-oriented application components made up of small independent services that communicate over well-defined APIs for notification and process queueing
- They make applications and apps faster to develop and easier to scale by small, self-contained teams of developers
 - **Microservices are about the design of software**
 - **Containers are about packaging software for deployment**





CONTAINER DEVELOPMENT VULNERABILITIES

- Unsecure images
 - Containers or microservices that have legacy code with known vulnerabilities
 - They derive from registries that are known to be suspect or have not been verified
- Container breakout (escape)
 - When malicious code or attackers leak from their confined container into the host operating system or hypervisor



CONTAINER DEVELOPMENT VULNERABILITIES

- Poisoned orchestration API
 - Unsecure Kubernetes (K8) environments leading to unauthorized access, data theft, or worse
- Denial-of-service
 - A particular container is inundated with too many requests, forcing it to buckle
 - It could possibly cascade into negatively affecting communication with other containers or microservices

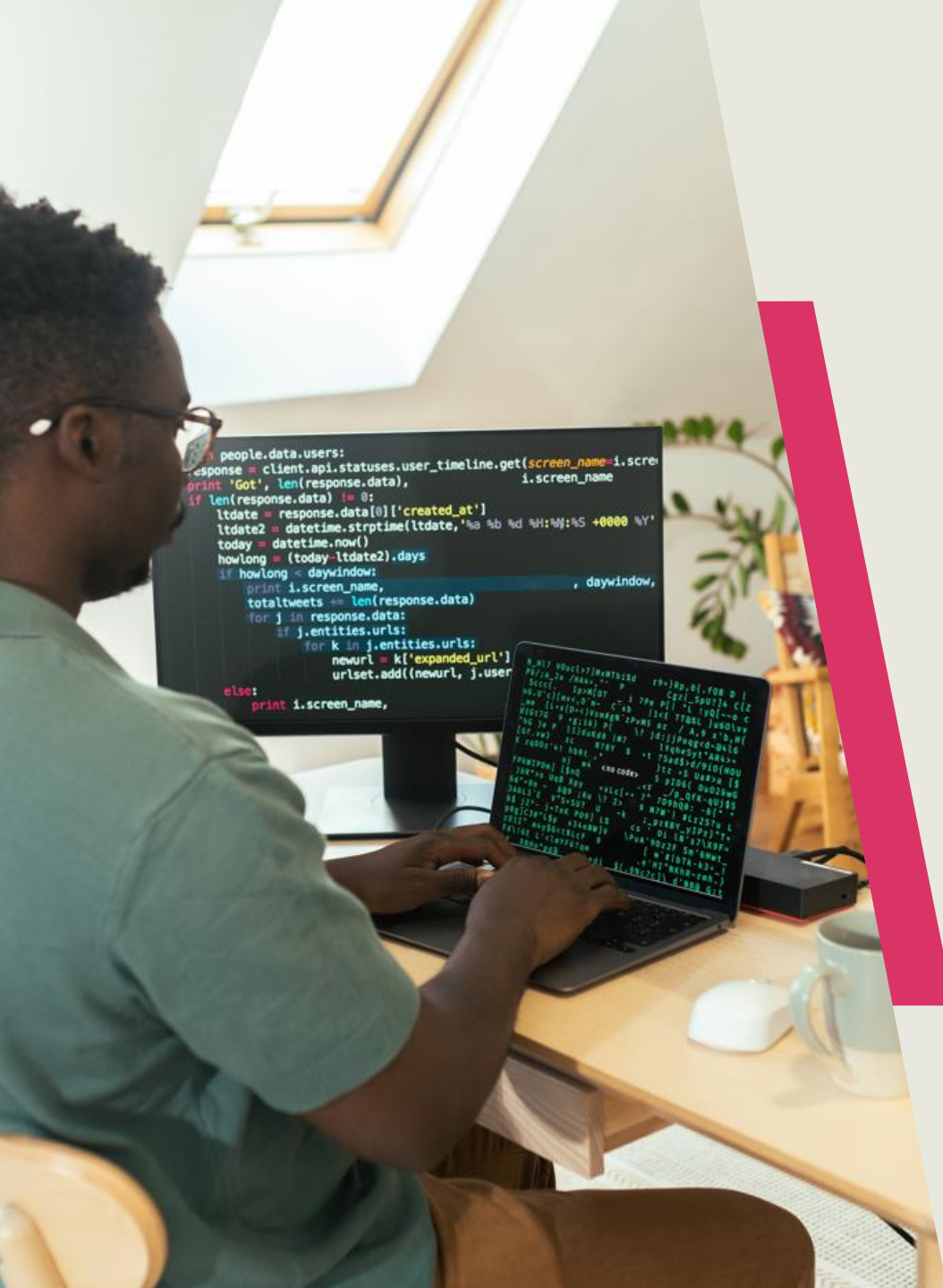
A person with dark hair and glasses is focused on their laptop. The image is stylized with glowing digital code and data patterns overlaid on the person's face and the laptop screen. The code includes terms like 'AbstractMap', 'constructor', 'Vx', 'backward', 'input, or throws', 'ReturnValue', and 'mutable V va'.

CONTAINER DEVELOPMENT VULNERABILITIES

- Untrusted images
 - Unreliable images can carry malicious code or hidden backdoors
- Lack of resource limiting
 - A poorly written container consumes too many processor, memory, or network resources
 - This could trigger application or system instability as other microservices on the same host system get starved

CONTAINER DEVELOPMENT VULNERABILITIES

- Misconfigured access controls
 - Failing to implement due diligence with least privilege and segregation of duties principles
- Insufficient isolation
 - At their core, containers, and microservices are designed to provide a certain degree of isolation – from the host system and other containers
 - Without solid configuration, this isolation shield issue can become problematic



CONTAINER SECURITY COUNTERMEASURES

- Use Minimal Base Images
 - Start with lightweight, trusted images (e.g., Alpine Linux) to reduce attack surface
- Avoid bloated or unverified images from public registries
 - Implement Image Signing & Scanning
- Digitally sign container images to verify integrity and origin
 - Scan images for vulnerabilities before deployment using tools like Clair
- Apply Runtime Security Controls
 - Use container runtime protection tools (e.g., Falco) to monitor behavior and detect anomalies
 - Enforce syscall restrictions and sandboxing





CONTAINER SECURITY COUNTERMEASURES

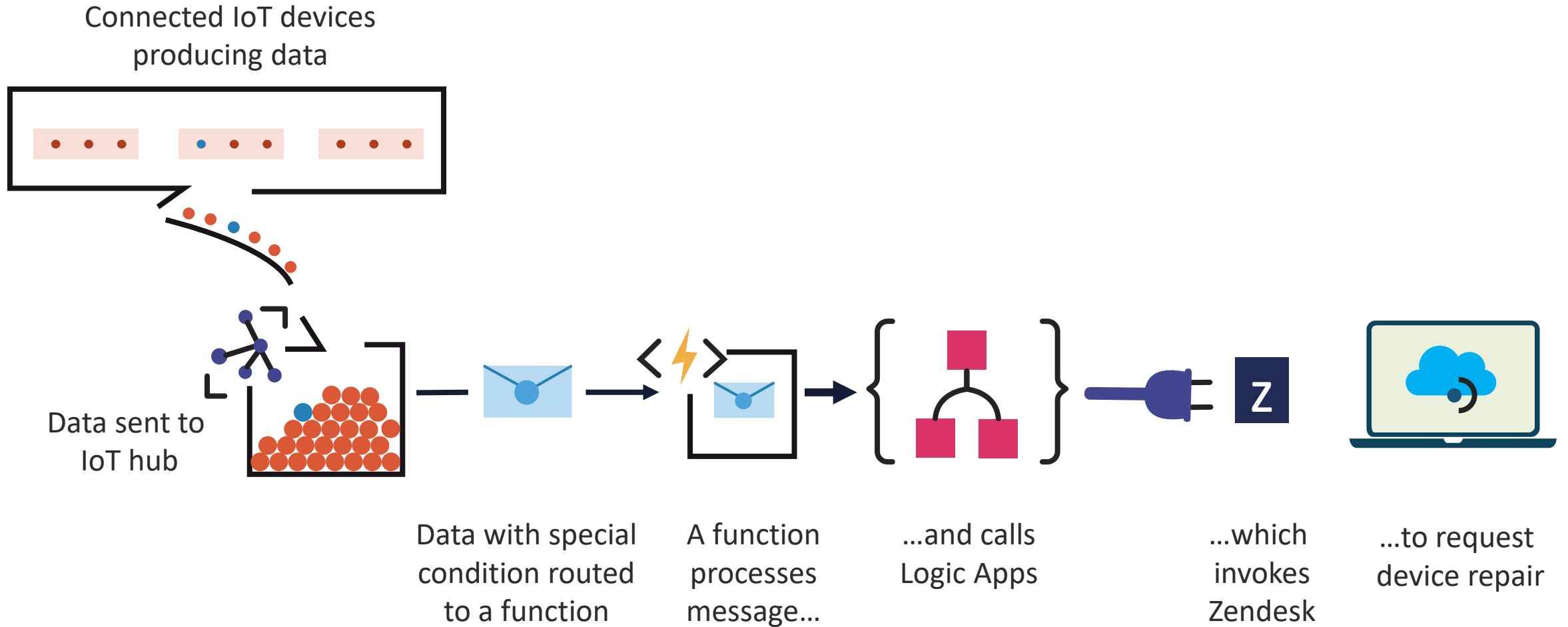
- Isolate Containers with Namespaces & Cgroups
 - Use Linux namespaces to isolate process trees, networking, and file systems
 - Apply control groups (cgroups) to restrict resource usage
- Secure Container Orchestration Platforms
 - Harden Kubernetes or Docker Swarm configurations
 - Use RBAC, network policies, and audit logging within orchestration layers
- Regular Patch & Update Cycles
 - Keep container images and orchestration tools up to date
 - Automate rebuilds and redeployments when base images or dependencies are patched
- Centralize logs from containers into SIEM platforms

SERVERLESS TECHNOLOGIES

- Modern serverless solutions leverage modern cloud infrastructures to emulate the network operating system (NOS) environment without the need for a Windows or Linux-based server
- These run code, manage data, and integrate applications, all without underlying servers
- They feature automatic scaling, built-in high availability, and a pay-for-use billing model to increase agility and optimize costs
 - **Functions, containers, and databases are common serverless solutions**

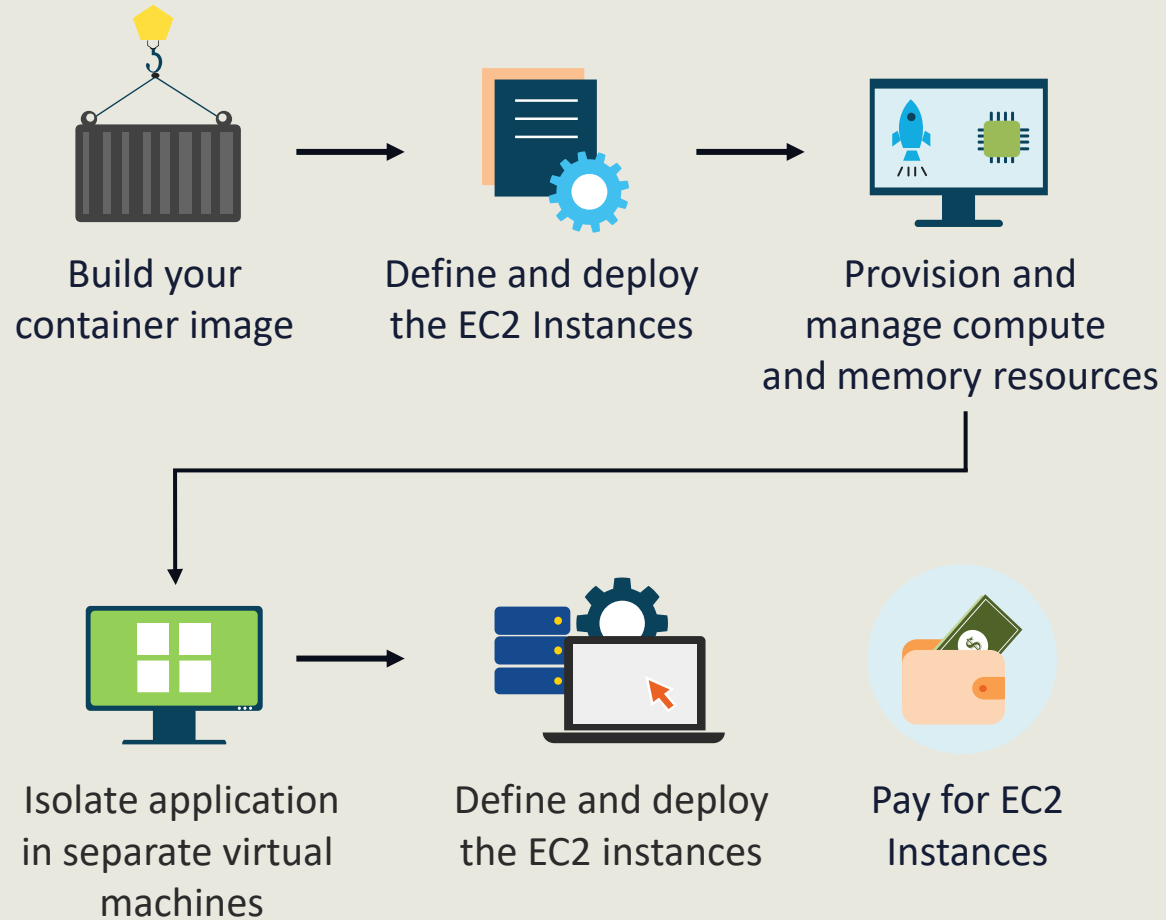


CASE STUDY: AZURE SERVERLESS FUNCTIONS

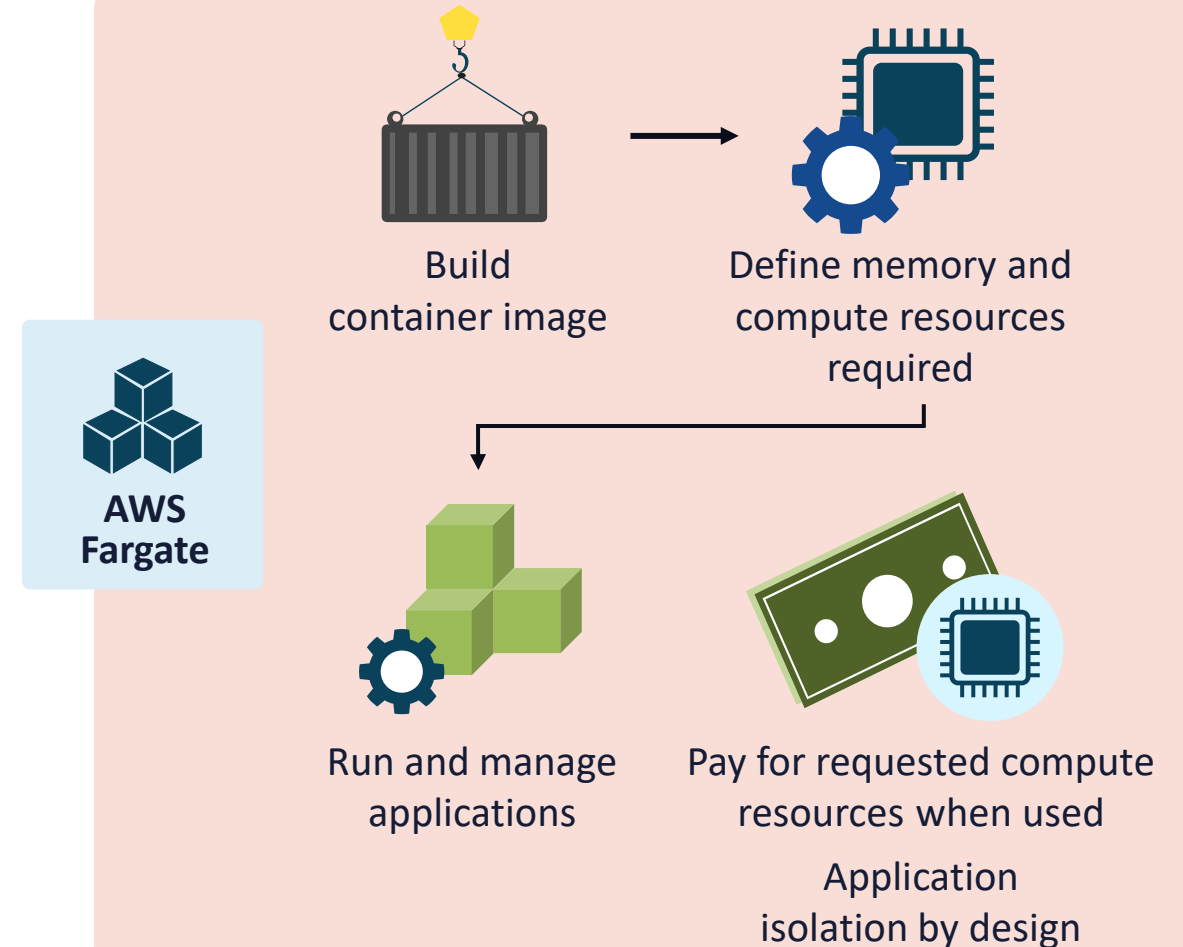


CASE STUDY: AWS FARGATE SERVERLESS CONTAINERS

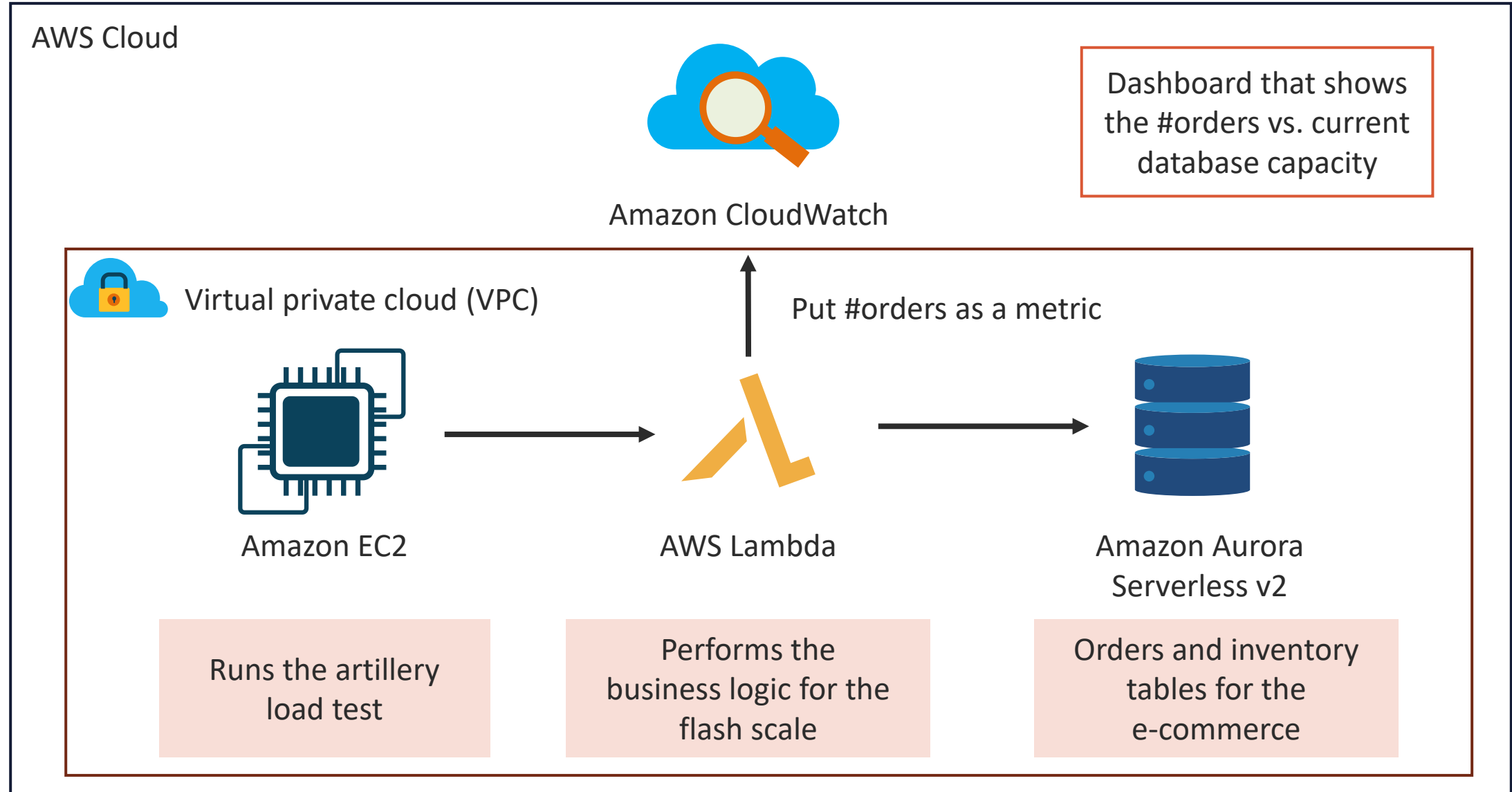
Without Fargate



With Fargate



CASE STUDY: AWS SERVERLESS DATABASE WITH AMAZON AURORA



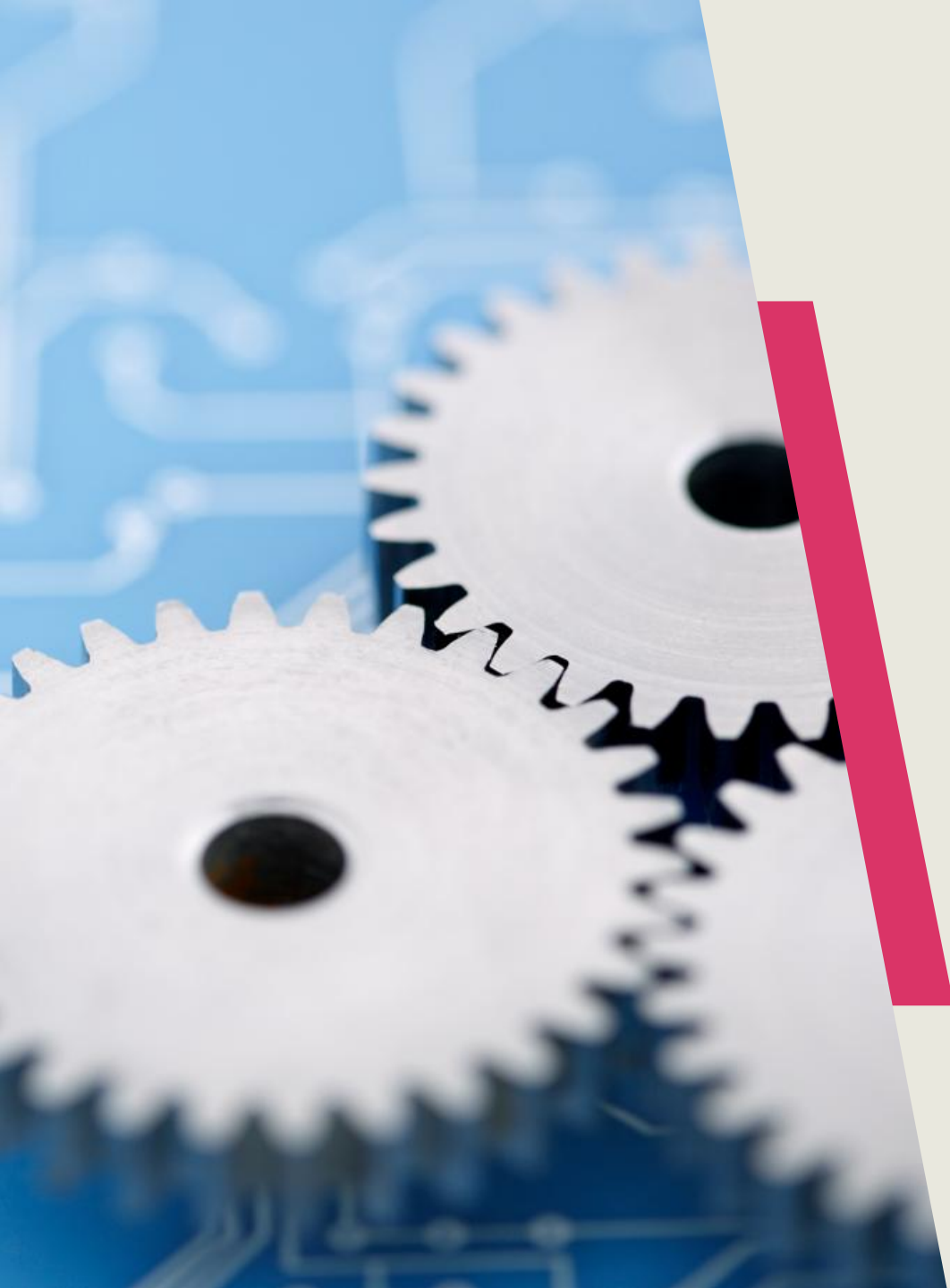
SERVERLESS VULNERABILITIES

- Lack of visibility and control of the cloud computing environment
- Absence of source code security
- Using obsolete or unpatched libraries, packages, and dependencies
- Forgetting to integrate the newest cryptographic protections into the code
- Too much information being disclosed in error messages

HIGH-PERFORMANCE COMPUTING (HPC)

- An HPC cluster is a set of independent computing systems, called compute nodes, which are interconnected via high-speed networks (100, 400 Gbps)
- An example of large-scale HPC would be a Dell solution with 9,216 CPU cores, and 1.28PB Power Scale Network File Storage (NFS)
 - These are usually purchased on a 1/3/5-year subscription from various vendors such as Dell or HP





HIGH-PERFORMANCE COMPUTING

- If customers decide to "rent" an HPC solution instead, they may use a CSP like AWS
 - Amazon Elastic Compute Cloud (EC2) Hpc6a instances offer 3rd Gen AMD EPYC processors
 - Elastic Fabric Adapter (EFA), powered by AWS Nitro, delivers low-latency 100 Gbps network bandwidth for internode communication
- Built for compute-intensive workloads such as:
 - Computational fluid dynamics (CFD)
 - Weather forecasting
 - Multiphysics* simulations
 - Large Language Modeling

NIST SP 800-223

- **High-Performance Computing Security: Architecture, Threat Analysis, and Security Posture**
- According to NIST: "An HPC system is designed to maximize performance, so its architecture, hardware components, software stacks, and working environment are very different from enterprise IT."
- Additionally, most HPC systems serve multiple projects and user communities, leading to more complex security concerns than those encountered in enterprise systems
- As such, security solutions must be tailored to the HPC system's requirements."



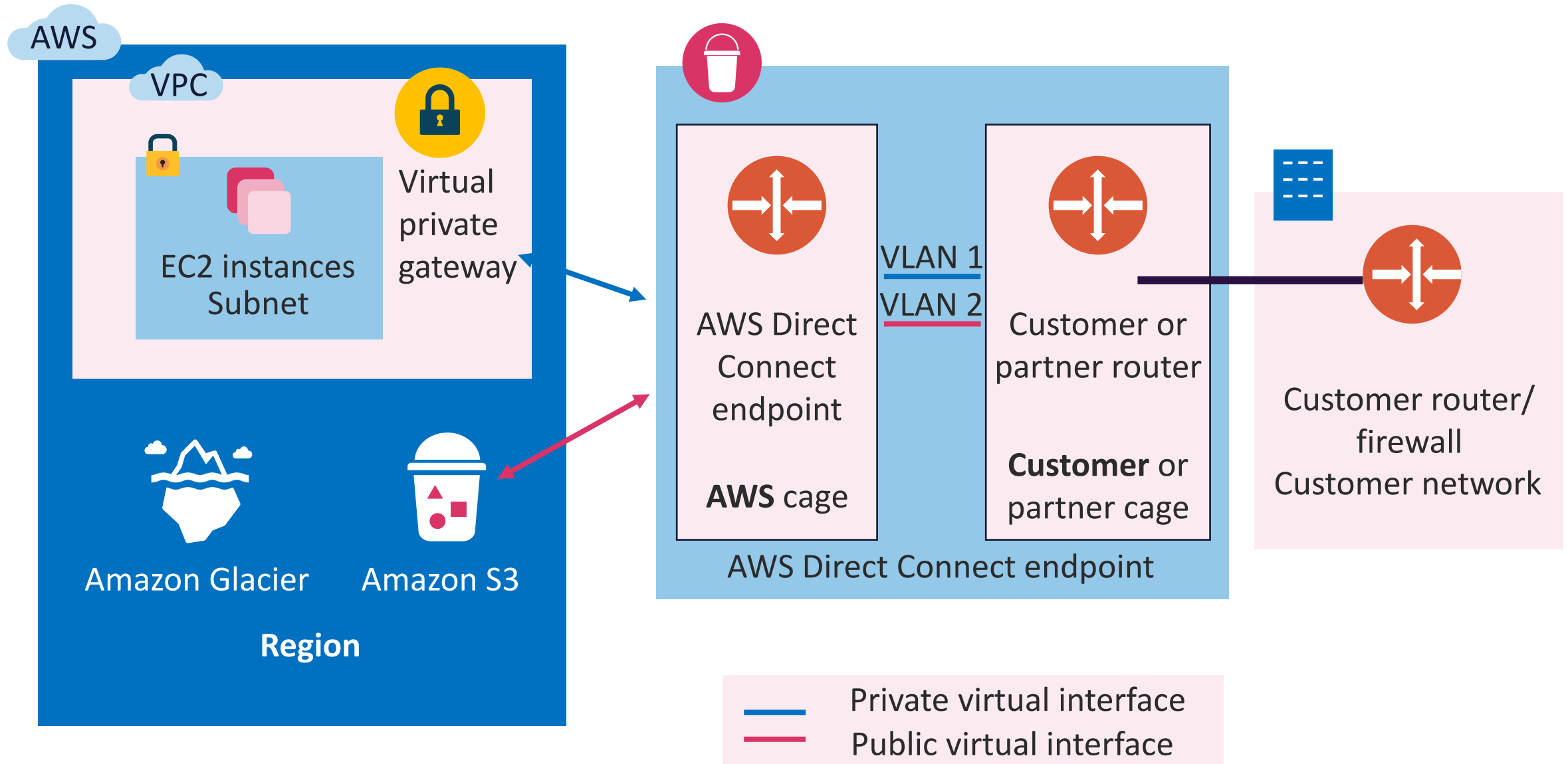
EDGE COMPUTING (FOG)



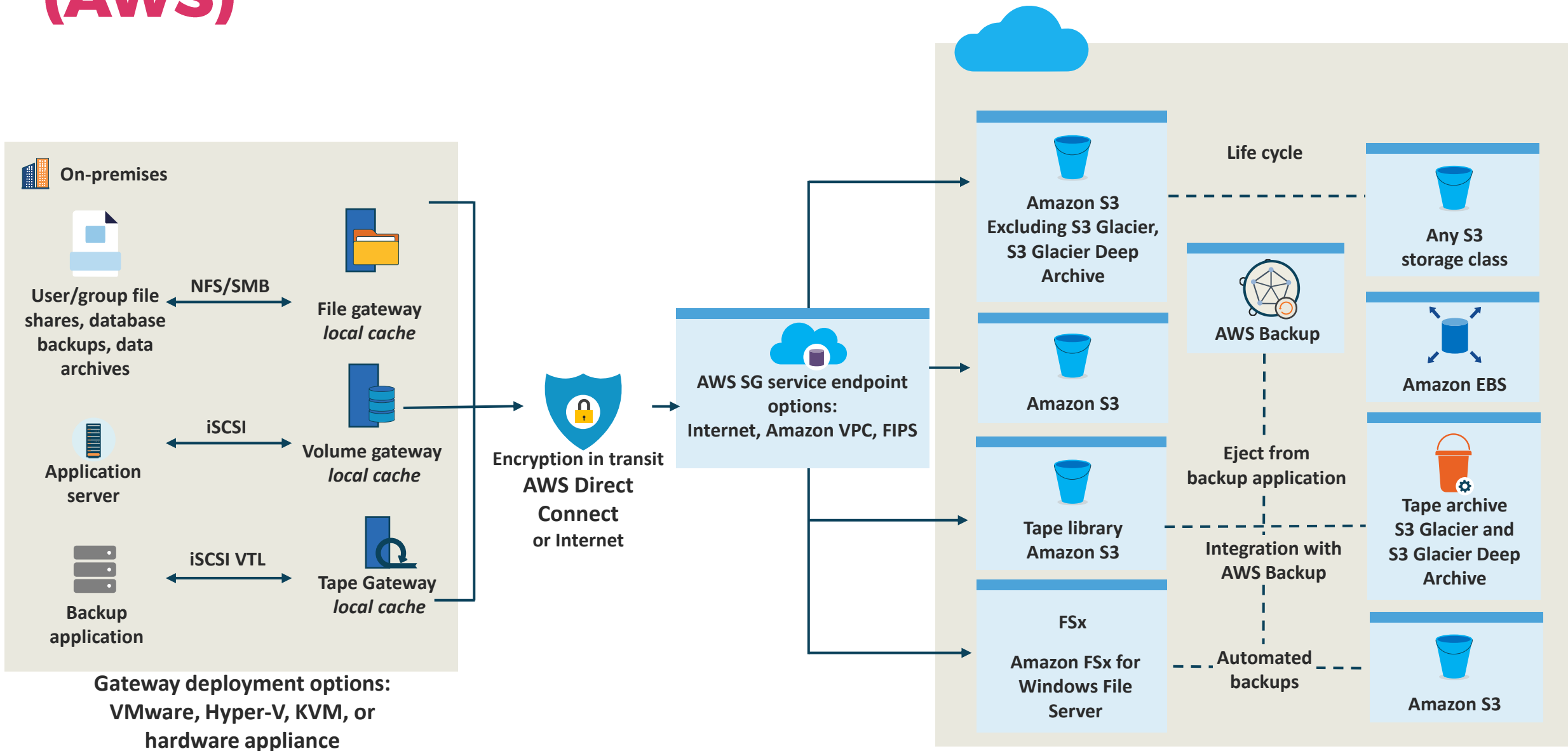
EDGE COMPUTING

- Edge computing is a distributed computing standard that brings compute services and data storage near to the customer location where it is
- The goal is to speed up response times and preserve bandwidth (**hybrid cloud and local zones**)
- Content delivery network (CDN) solutions place cached versions of content (often in elastic Redis in-memory storage clusters) at metro area edge locations

DIRECT CONNECT SUPPORTS EDGE (FOG) COMPUTING

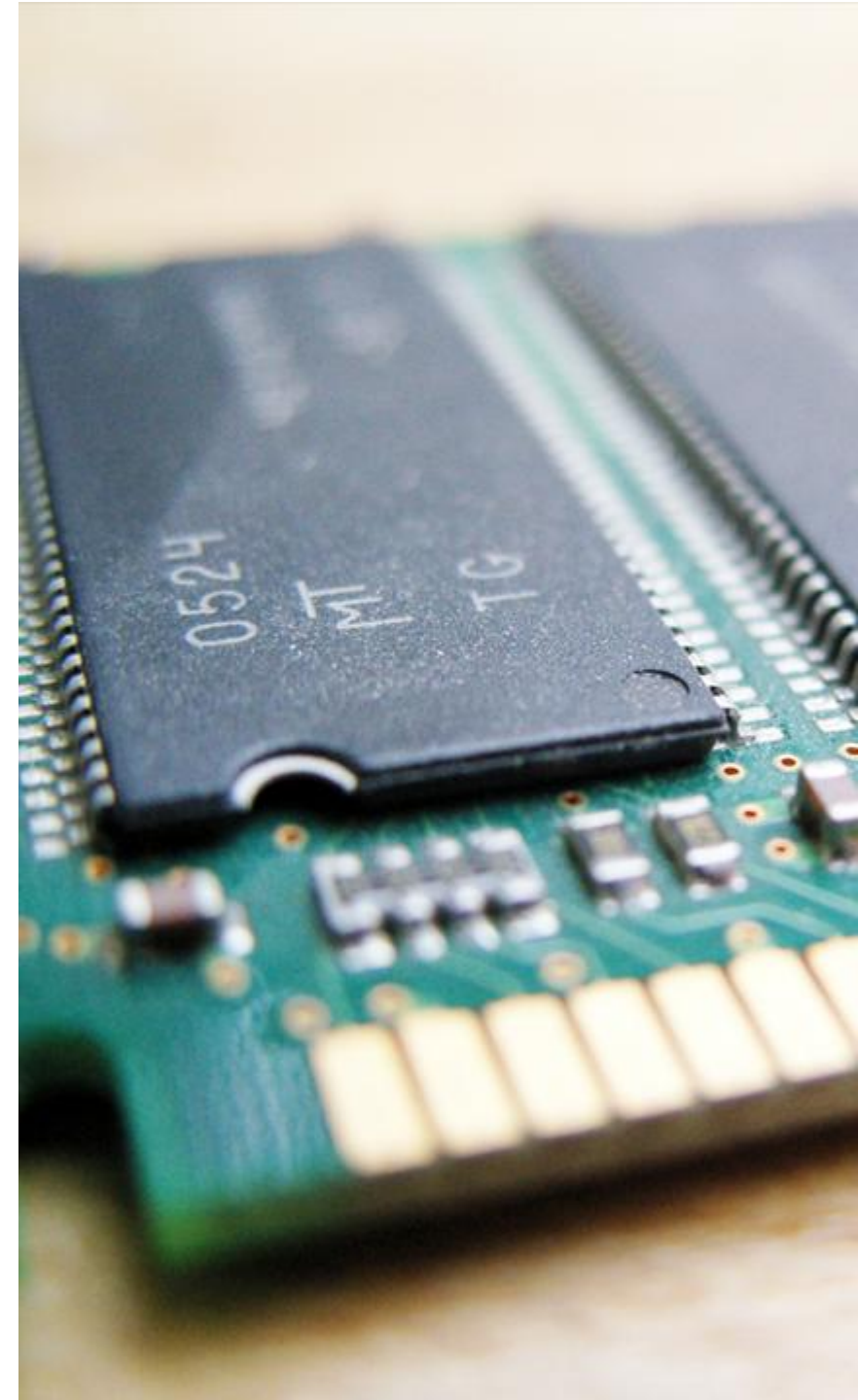


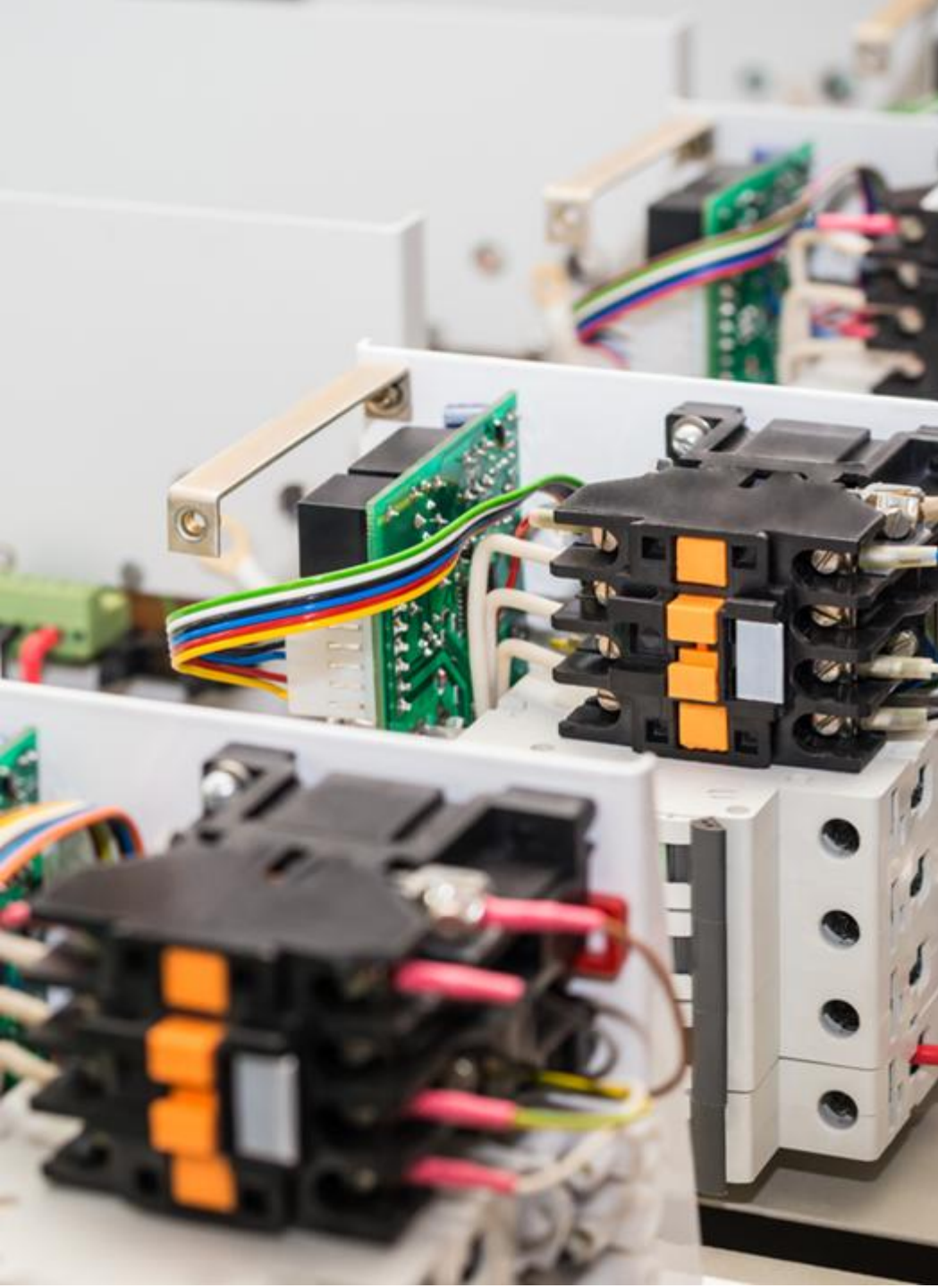
EDGE COMPUTING WITH STORAGE GATEWAYS (AWS)



TRUSTED EXECUTION ENVIRONMENTS (TEE) – MEMORY PROTECTION

- In a nutshell, Trusted Execution Environments (TEEs) are CPU-encrypted isolated private enclaves inside the memory
- The goal is to protect data-in-use at the hardware level
- Unauthorized subjects cannot remove, change, or add more data while the sensitive data is inside an enclave
- A TEE can greatly boost mobile (iOS SEP) and cloud data security by isolating sensitive functions and providing a secure environment for analyzing data





TRUSTED PLATFORM MODULE (TPM)

- A Trusted Platform Module 2.0 (TPM 2.0) is a hardware component that generates and stores cryptographic keys, secrets, and certificates
- It enhances the system security by preventing unauthorized access and tampering (confidentiality and integrity)
- To use a TPM, a software library that communicates with the device and a user-friendly API is needed



TRUSTED PLATFORM MODULE 2.0 TYPES

- **Discrete** TPMs are the most secure dedicated chips that deliver TPM capabilities in their own tamper-resistant semiconductor package
- **Integrated** TPMs are part of another chip
- **Firmware** TPMs (fTPMs) are firmware-based UEFI solutions that run in a CPU's TEE
- **Virtual** TPMs (vTPMs) are hypervisor solutions in isolated execution environments
- **Software** TPMs are software emulators of TPMs that run in an operating system and are the least secure option

TPM 2.0 ENCRYPTION AND DECRYPTION

- The **PC Client Platform TPM Profile (PTP) Specification** requires:
 - SHA-1 and SHA-256 for hashes
 - RSA, ECC using the NIST P-256 curve for public-key cryptography and asymmetric digital signature generation and verification
 - HMAC for symmetric digital signature generation and verification
 - 128-bit AES for symmetric-key algorithm
 - ECC-based 256-bit curve is optional for the TCG PC Client Platform PTP Specification
- **Homomorphic encryption is being introduced in cloud computing environments**



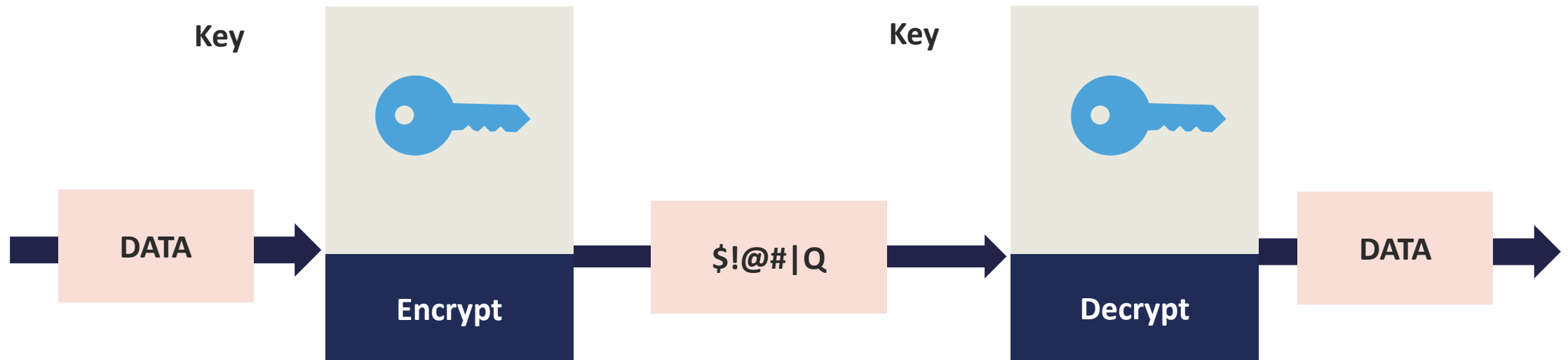
CRYPTOGRAPHIC SOLUTIONS & CRYPTANALYTIC ATTACKS

Objectives

- Learn about cryptographic methods and life cycle
- Describe key management, digital signatures, certificates, and public key infrastructure
- Explain brute force, implementation, side-channel, an on-path attacks against cryptosystems
- Learn about Kerberos exploitation
- Understand ransomware attacks

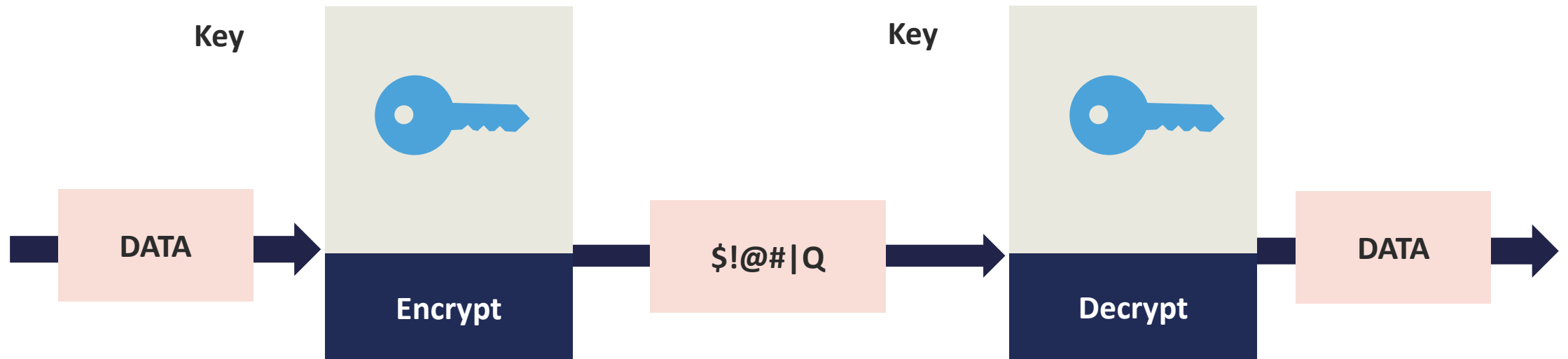
SYMMETRIC KEY CRYPTOGRAPHY

- The same secret key is used for encryption and decryption
- The secret key must be shared between sender and receiver securely
- Strength is related to management, size of keys, and how they are shared
- Keys are typically from 40 to 512 bits in length
- Longer keys are less susceptible to a successful brute force attack



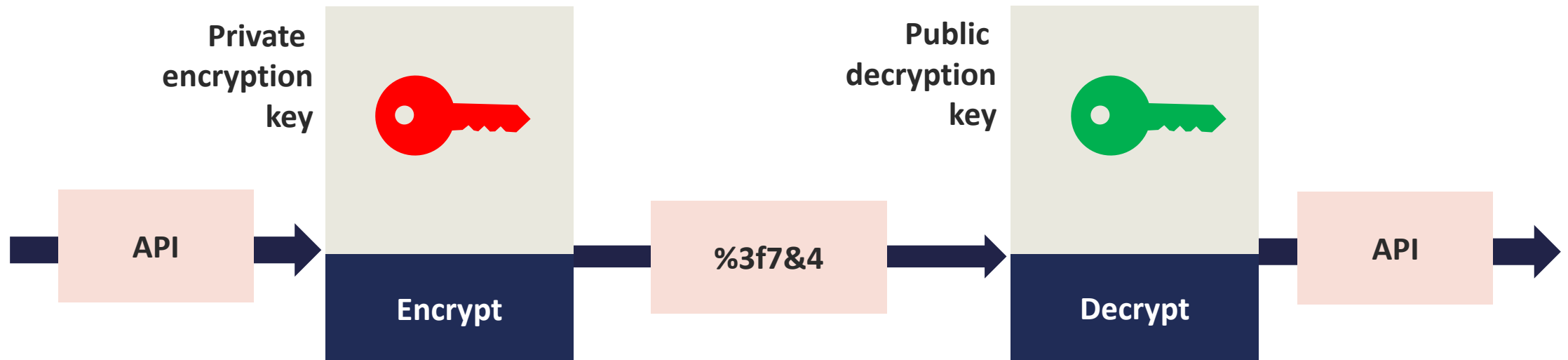
SYMMETRIC KEY CRYPTOGRAPHY

- Symmetric algorithms deliver wire-speed encryption
- They are commonly used for bulk data encryption (e.g., VPNs, data at rest, CMKs at CSPs)
- They utilize strong confusion and diffusion techniques
- Algorithms can be accelerated by hardware security module (HSM)
- Symmetric can use both stream and block ciphers (block are most common)



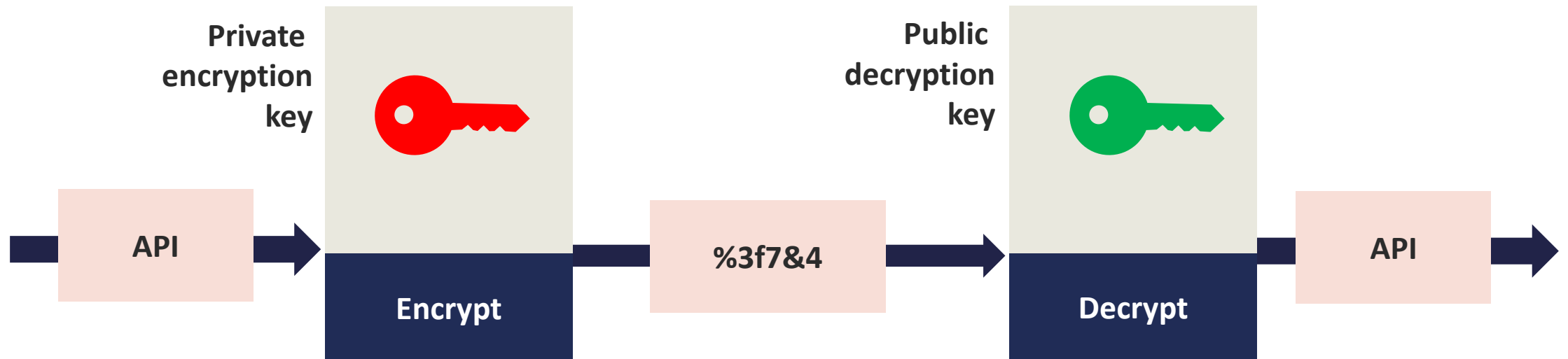
ASYMMETRIC KEY CRYPTOGRAPHY

- Different keys are used for encryption and decryption
- They are generated together and are mathematically related
- The public key is shared with multiple parties
- The private key must be kept private and confidential by the owner or custodian
- Keyspace ranges from 512 to 4,096 bits in length



ASYMMETRIC KEY CRYPTOGRAPHY

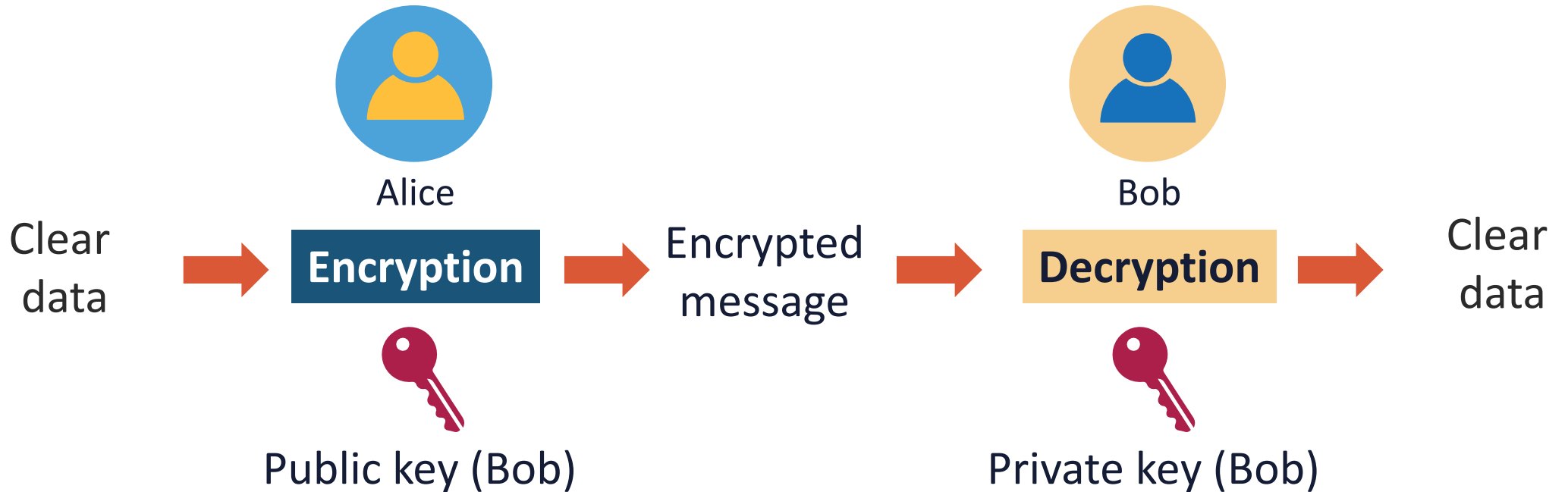
- Asymmetric cryptosystems are slower and not suitable for bulk data operations
- The design is based on factoring the product of large prime numbers
- Key management is simpler and more secure (PKI and HSM)
- They are best suited for digital signatures and session key exchange or agreement protection services (RSA, DSA, Elliptic curve DSA, PGP/GPG, DHKE, ECDHE)



ASYMMETRIC CRYPTOSYSTEMS FOR PRIVACY



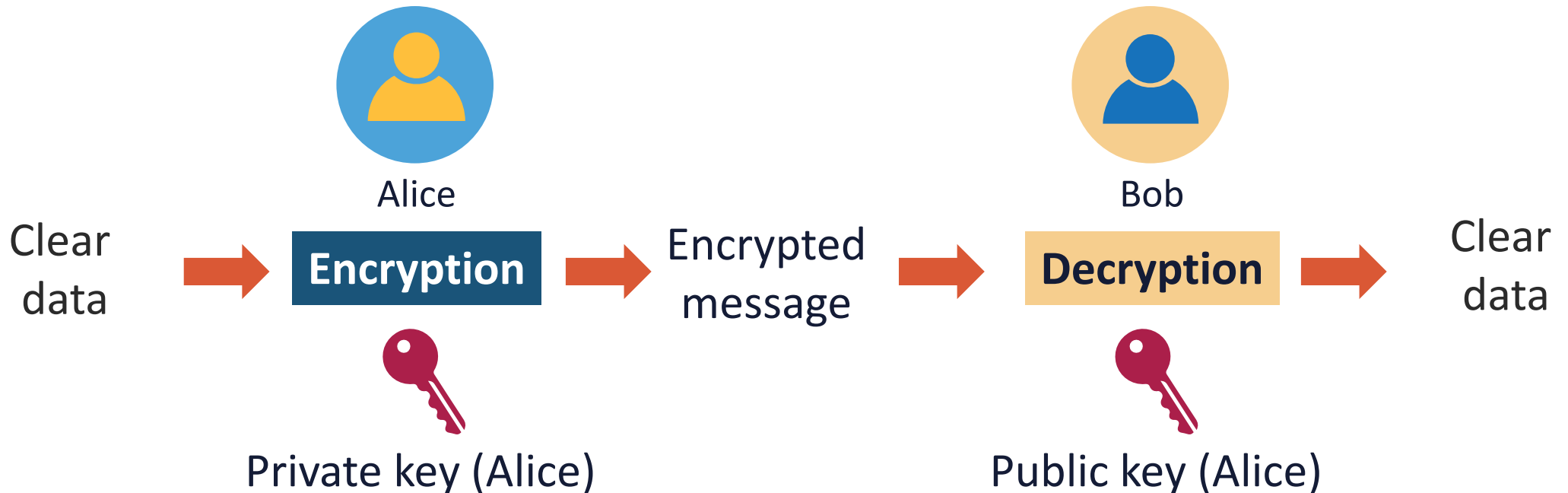
Privacy (confidentiality) = Alice encrypts with Bob's public key, then Bob decrypts with his private key



ASYMMETRIC CRYPTOSYSTEMS FOR AUTHENTICATION

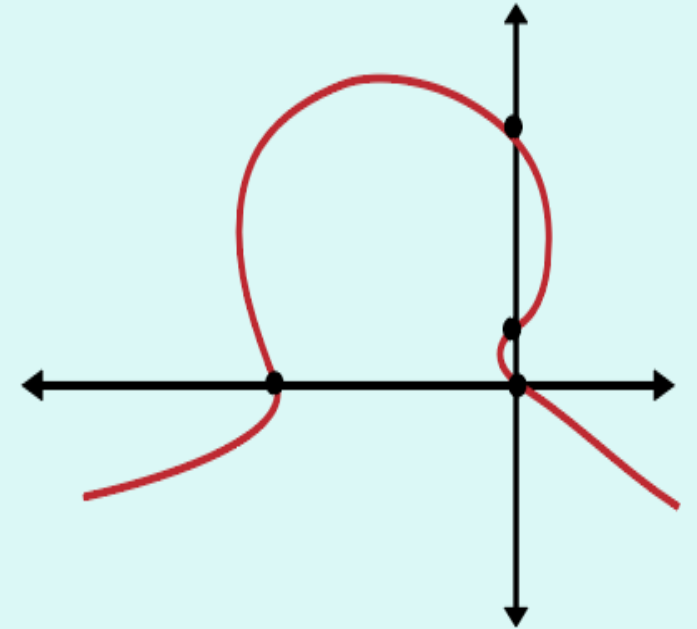


Origin authentication = Alice encrypts with her private key, then Bob decrypts with Alice's public key



ELLIPTIC CURVE CRYPTOGRAPHY

- Elliptic curve cryptography (ECC) involves rich mathematical functions based on points on a curve
- The algorithm computes discrete logarithms of elliptic curves, which is different from calculating discrete logarithms in a finite field
 - The smaller and more efficient keys offer exceptional speed and strength (e.g., a 256 elliptic curve key is equivalent to a 3,072 normal key)
- It is commonly used in digital signatures, key distribution, and encryption
- It is an excellent choice for mobile and Internet of Things (IoT) devices





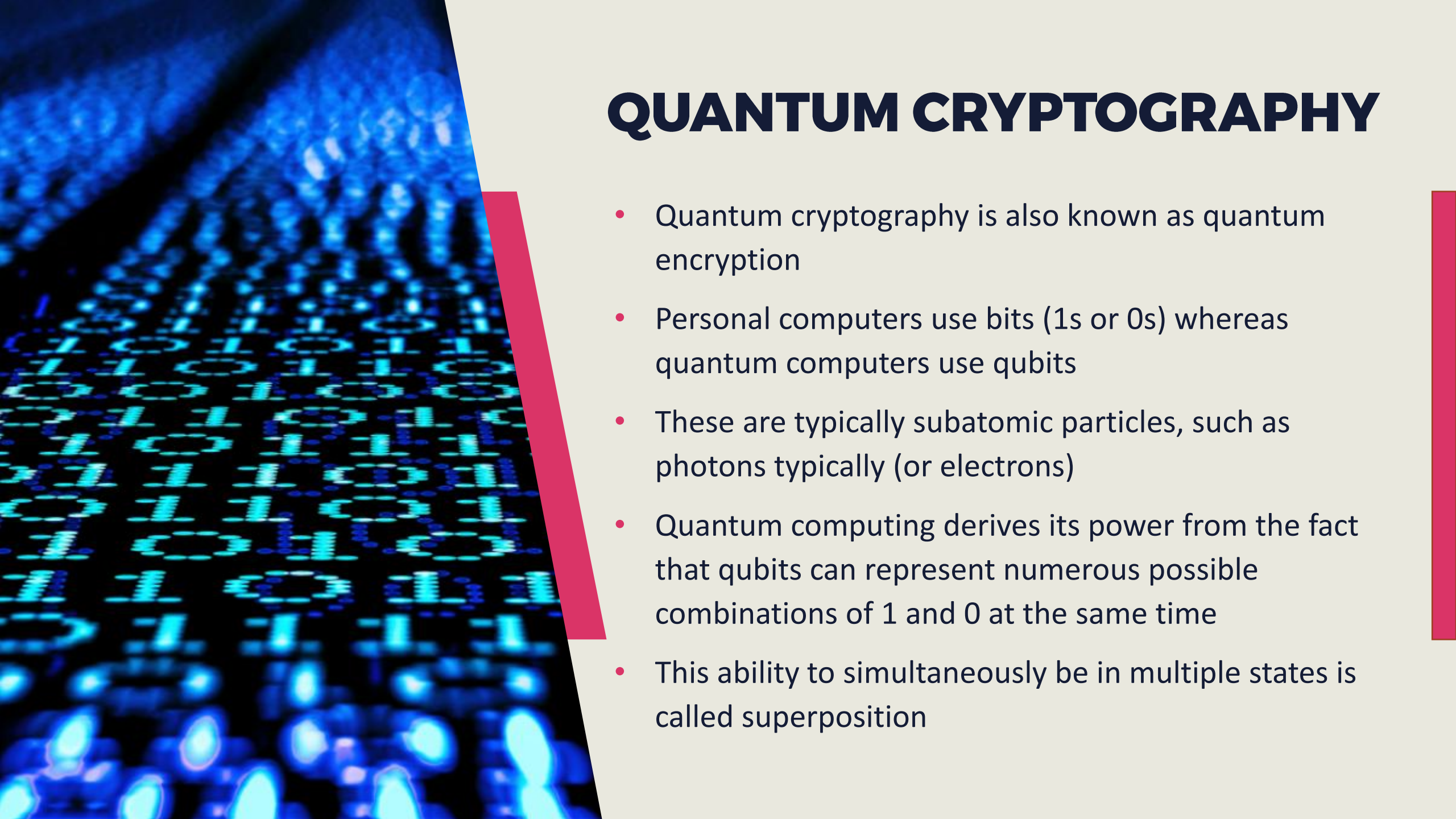
ELLIPTIC CURVE DIGITAL SIGNATURE ALGORITHM (ECDSA)

- It is a cryptographically secure digital signature system based on elliptic curve cryptography
 - It leverages the math of the cyclic groups of elliptic curves over finite fields and on the difficulty of the elliptic-curve discrete logarithm problem
- It is popular in the field of digital signatures built on the complexity of calculating a private key from which a public key is based
 - ECDSA is also used by Bitcoin to assure that cryptocurrency can only be used by the lawful owner(s)

ELLIPTIC CURVE DIFFIE-HELLMAN (ECDH)

- ECDH is a key agreement protocol that enables two entities, each having an elliptic curve key pair, to generate a shared secret over an unsecure channel like the public Internet
- The key (or derived key) can then be used to encrypt successive communications using a symmetric-key cipher
- ECDHE is a variant of the Elliptic Curve Diffie-Hellman (ECDH) protocol that uses ephemeral keys—temporary key pairs generated for each session—to establish a shared secret between two parties over an insecure channel

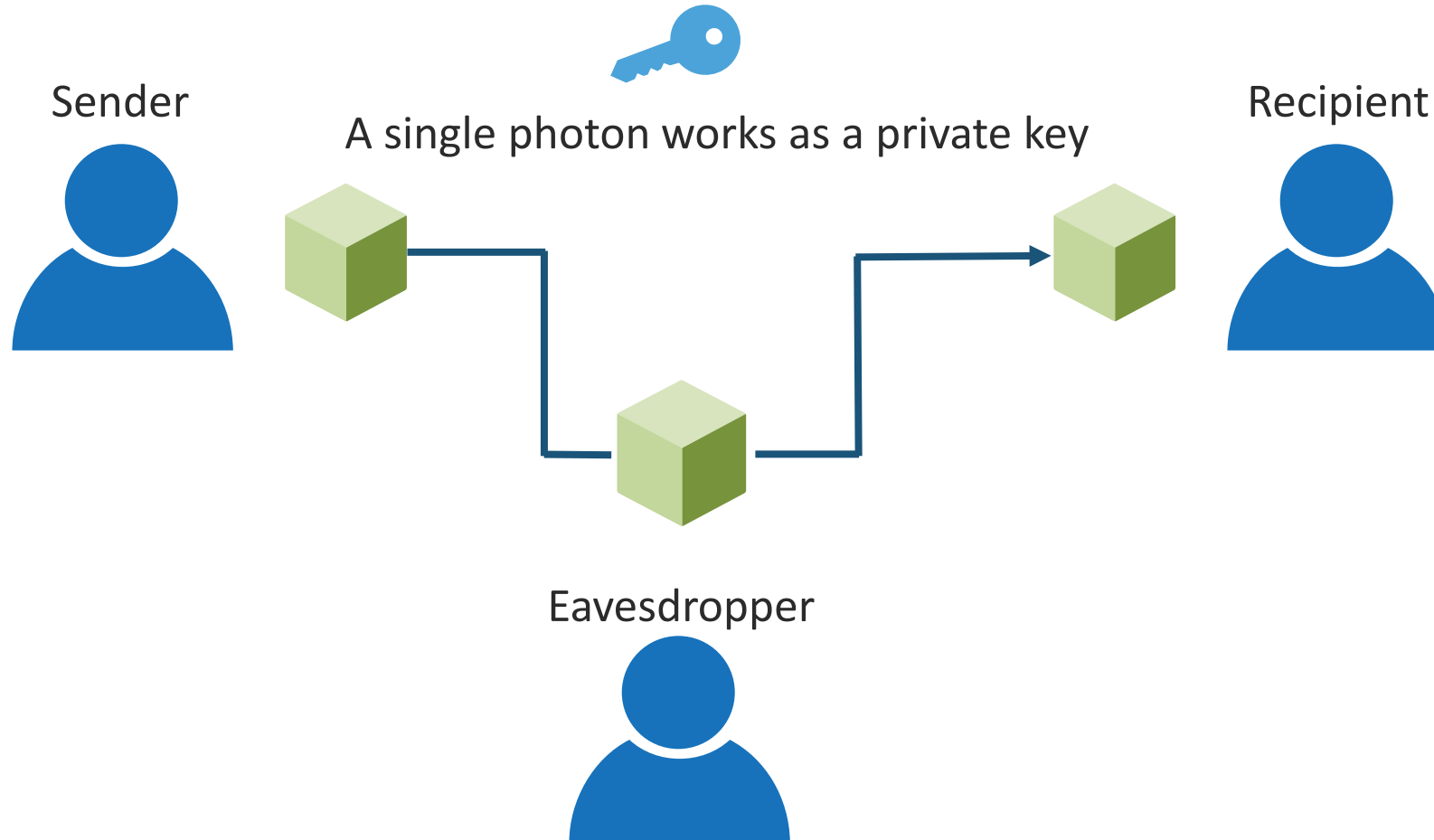




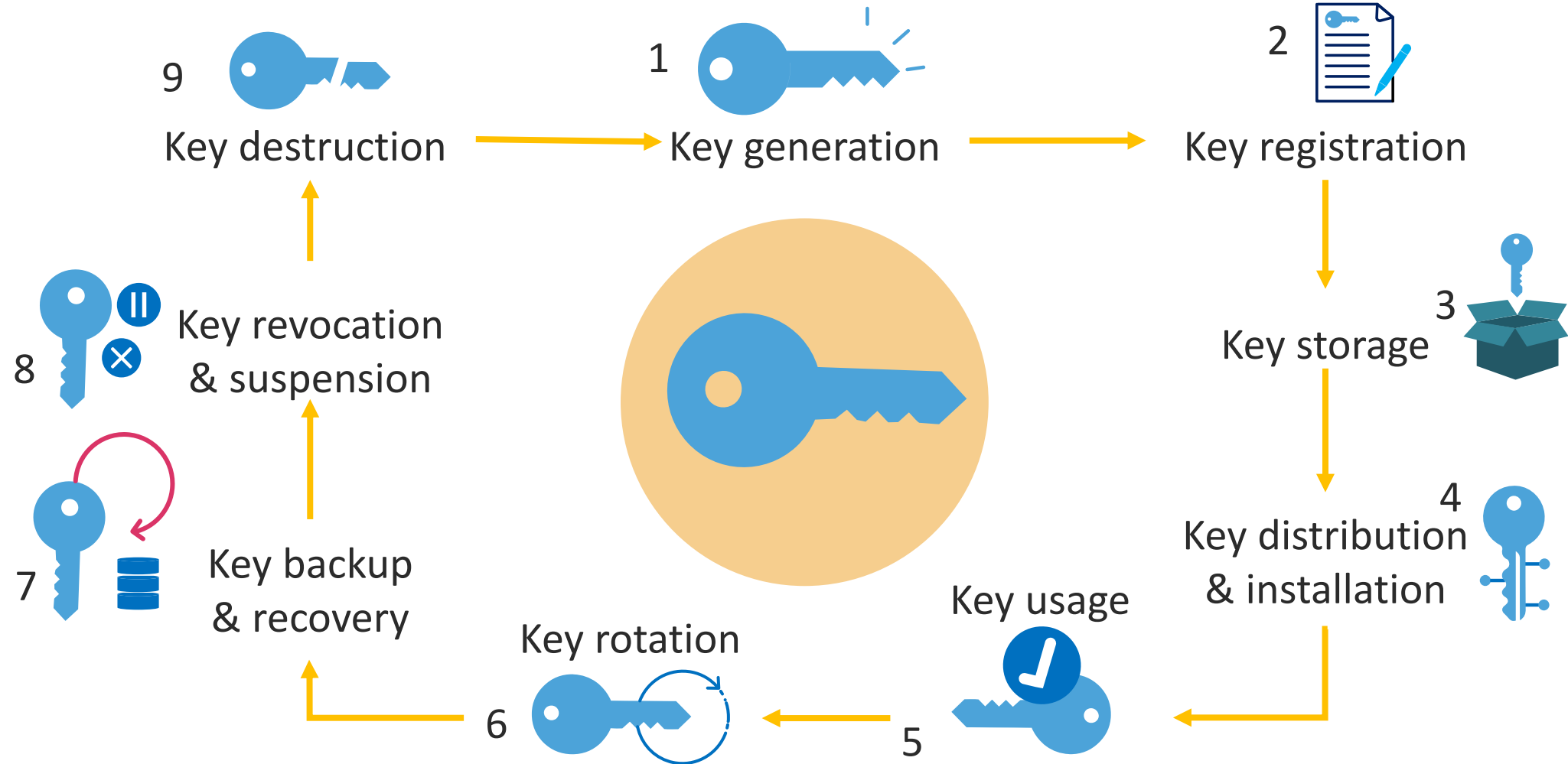
QUANTUM CRYPTOGRAPHY

- Quantum cryptography is also known as quantum encryption
- Personal computers use bits (1s or 0s) whereas quantum computers use qubits
- These are typically subatomic particles, such as photons typically (or electrons)
- Quantum computing derives its power from the fact that qubits can represent numerous possible combinations of 1 and 0 at the same time
- This ability to simultaneously be in multiple states is called superposition

QUANTUM CRYPTOGRAPHY SYSTEM



CRYPTOGRAPHIC LIFE CYCLE





CRYPTOGRAPHIC KEY LIFE CYCLE: GENERATION

- Keys can be generated using several different modalities:
 - Software program (RSA 2048) generating certificates
 - A key management system (for example AWS KMS)
 - Hardware security modules
 - Trusted third party (TTP)
- Each key should have a key strength (in bits) to offer sufficient protection for the entire useful lifetime of the associated data – with the ability to endure attacks during this lifetime

CRYPTOGRAPHIC KEY LIFE CYCLE: REGISTRATION

- Keys can be associated with accounts, users, systems, code, and applications and registered in a repository like an HSM or KMS
- If keys are tagged or labeled with key/value pairs or other unique identifiers, this metadata must be protected as well
 - AWS uses a pseudorandom key identifier (Access Key ID) for its secret access keys used with the command line interface (CLI) or software development kit (SDK)





CRYPTOGRAPHIC KEY LIFE CYCLE: STORAGE

- The key storage database (which will have a master key for encryption) next stores the keys along with all extensible attributes
 - Qualities include things like name, activation date, size, and instance
- A key can be activated upon creation or set to be activated later either automatically or manually
- Keys are often stored in the same system that generated them as mentioned

CRYPTOGRAPHIC KEY LIFE CYCLE: DISTRIBUTION AND INSTALLATION (LOADING)



- The goal of the deployment and install (loading) stage is to put the new key into a secure cryptographic device, either manually or electronically
- This is a critical phase for key security and should only be done by highly-privileged and audited personnel when conducted manually (the most common method)
- Governance such as the Payment Card Industry Data Security Standard (PCI-DSS) now mandates that the key usage be equally secured along with encrypting the key material

CRYPTOGRAPHIC KEY LIFE CYCLE: USAGE (UTILITY)

- As with any type of data, keys should have utility, use cases, and meaning or they should not be generated
- The key management system should let activated keys be claimed by authorized users, systems, and apps for encryption or decryption processes, or message authentication code (MAC) creation and authentication





CRYPTOGRAPHIC KEY LIFE CYCLE: ROTATION

- Key managers should replace, and rotate keys, automatically based on an established schedule (according to the key's expiration date or life cycle)
 - This should also occur when compromise is suspected or a device with a certificate is lost or stolen
- When replacing keys, it is common to re-encrypt all stored data under the new key
- Rotating keys can be difficult overhead since it can entail additional procedures and protocols, which may include correspondence with third parties in public-key systems

CRYPTOGRAPHIC KEY LIFE CYCLE: BACKUP AND RECOVERY

- Sometimes a key can be lost during use (like equipment failure or forgotten credentials)
- A secure backup copy should be made available from media such as external media (CD, USB fob, FireWire) or using an existing traditional backup solution
- When a symmetric key or an asymmetric private key is being backed up, it must be encrypted before being stored
- **Backup may be referred to as "Archival"**
- The recovery and restoration process must be tested on a regular or automated basis



CRYPTOGRAPHIC KEY LIFE CYCLE: REVOCATION AND SUSPENSION



- The most common example of this process is when a device storing an X.509v3 certificate is lost or stolen
- A public key infrastructure (PKI) relies on the Certificate Revocation Lists (CRL) or an Online Certificate Status Protocol (OCSP) database to revoke the public/private key pair
- Keep it mind that a 'suspended' public key does not populate the list or database
 - This may be done for a leave of absense or temporary sabbatical of a subject

CRYPTOGRAPHIC KEY LIFE CYCLE: DESTRUCTION

- The final phase is the end of the key's life cycle, where all instances, or just specific instances, are eliminated
- This may occur according to an archive policy (7 to 10 years)
- Recovery of that key may be possible, depending upon the method used



KEY DESTRUCTION (DISPOSITION)

Key Deletion

Weakest: Typically removes only the reference or pointer to the key (e.g., directory entry), not the actual key material

Risk: Data may still be recoverable using forensic tools

Key Termination

Intermediate: Refers to ending a session or key lifecycle, often by revoking access or disabling use

Risk: May not physically remove the key from memory or storage unless paired with secure erasure

Key Termination

Strongest: Physically or cryptographically destroying the key material so it cannot be recovered

Methods include overwriting, degaussing, shredding, cryptoshredding



KEY MANAGEMENT BEST PRACTICES

- The cryptographic and key management algorithms being used by an application should start by knowing the objective utility of the application
- A thorough analysis of the use case of the application must be performed to determine the optimal key management method
- Use the most robust and pertinent suite of algorithms that are agreed upon between clients and servers, peers, and distributed systems
- Leverage Vendor-based or Cloud-based KMS or CloudHSM



KEY MANAGEMENT BEST PRACTICES

- Keys stored in memory for a long time can become "burned in" so this must be mitigated by splitting the key into components that are frequently updated (NIST SP 800-57)
- By implementing temporary ephemeral keys, entities can achieve **perfect forward secrecy** protection
 - This means that a compromise of the service or application's long-term signing key does not compromise the confidentiality of past sessions



KEY MANAGEMENT BEST PRACTICES

- According to NIST, a distinct key should only be used for one purpose
 - Including encryption, authentication, key wrapping, random number generation, or digital signatures
- Valid reasons include:
 - Using the same key for two different processes may weaken the security provided by one or both use cases
 - Restricting the use of a key limits the damage that could be done if the key is compromised
 - Some usages of keys can interfere with each other

TRUST ANCHORS

- Authentication of endpoints is typically performed with pre-shared symmetric keys, trusted certificates, and **trust anchors**
- Trust anchors are authoritative entities in cryptographic systems such as public keys and their associated information, Certification authorities (CA) in a PKI, or known, trusted entities or values in a security system

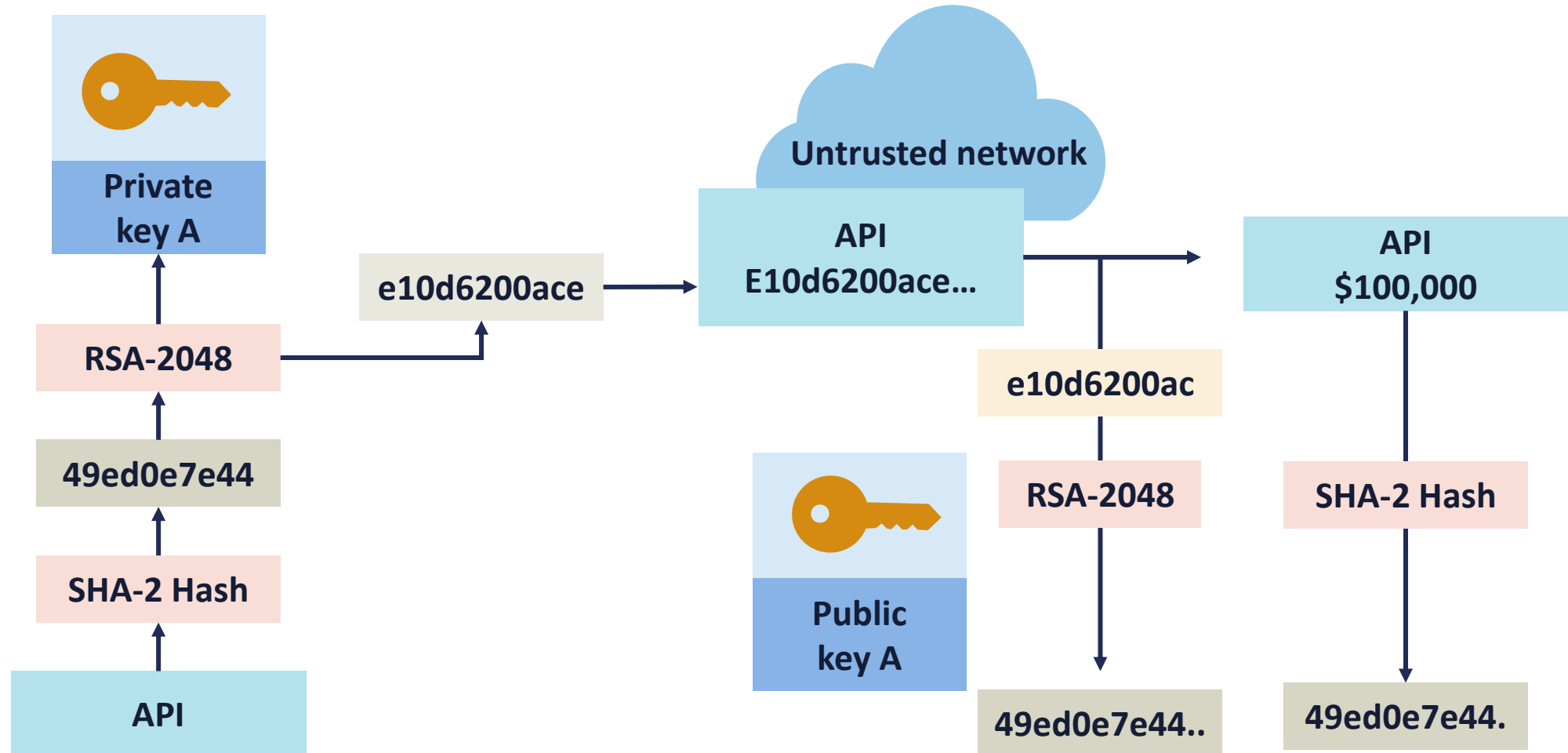


DIGITAL SIGNATURES

- Digital signatures are a form of electronic signature that was designed to replace and/or supplement a physical handwritten signature used to identify principals and protect information in digital messages or documents
- This can include email, credit card, online transaction, or some form of digital document
- They create a virtual fingerprint that is unique to an entity and are often more secure than other forms of physical and electronic signatures

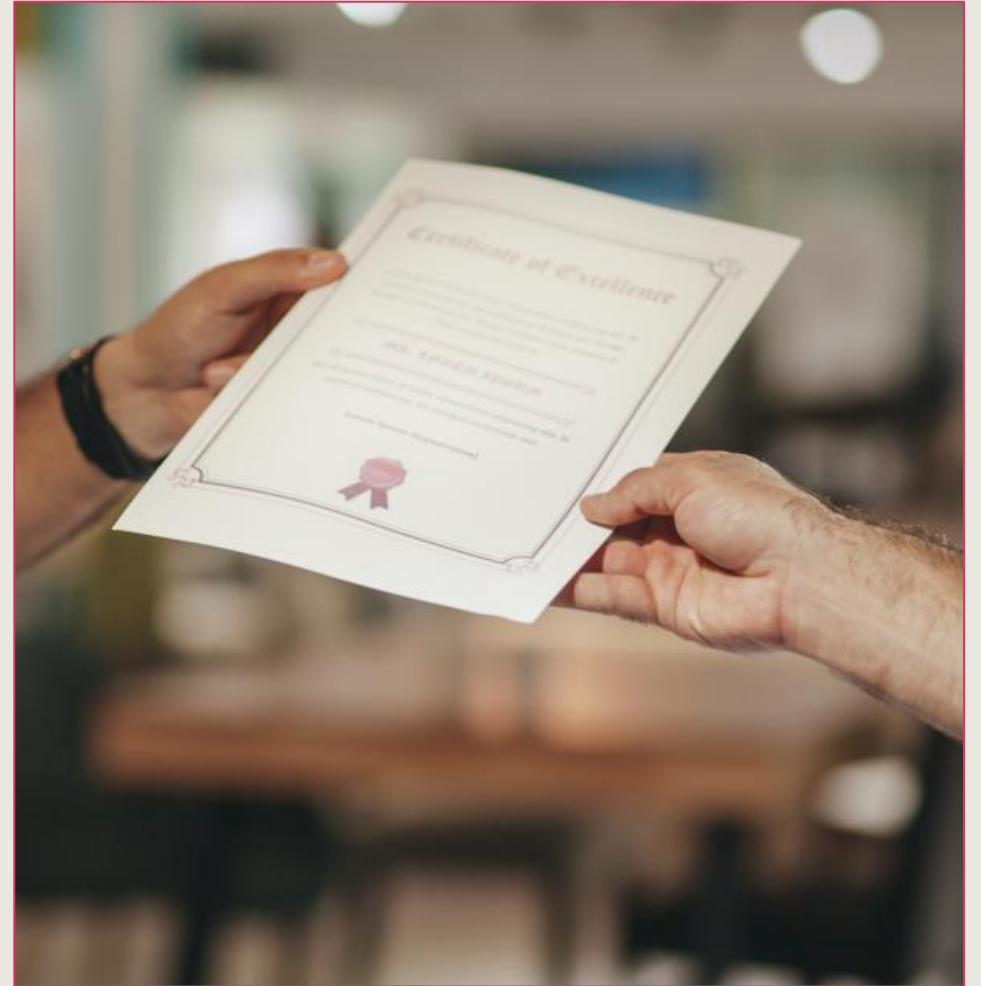


DIGITAL SIGNATURES

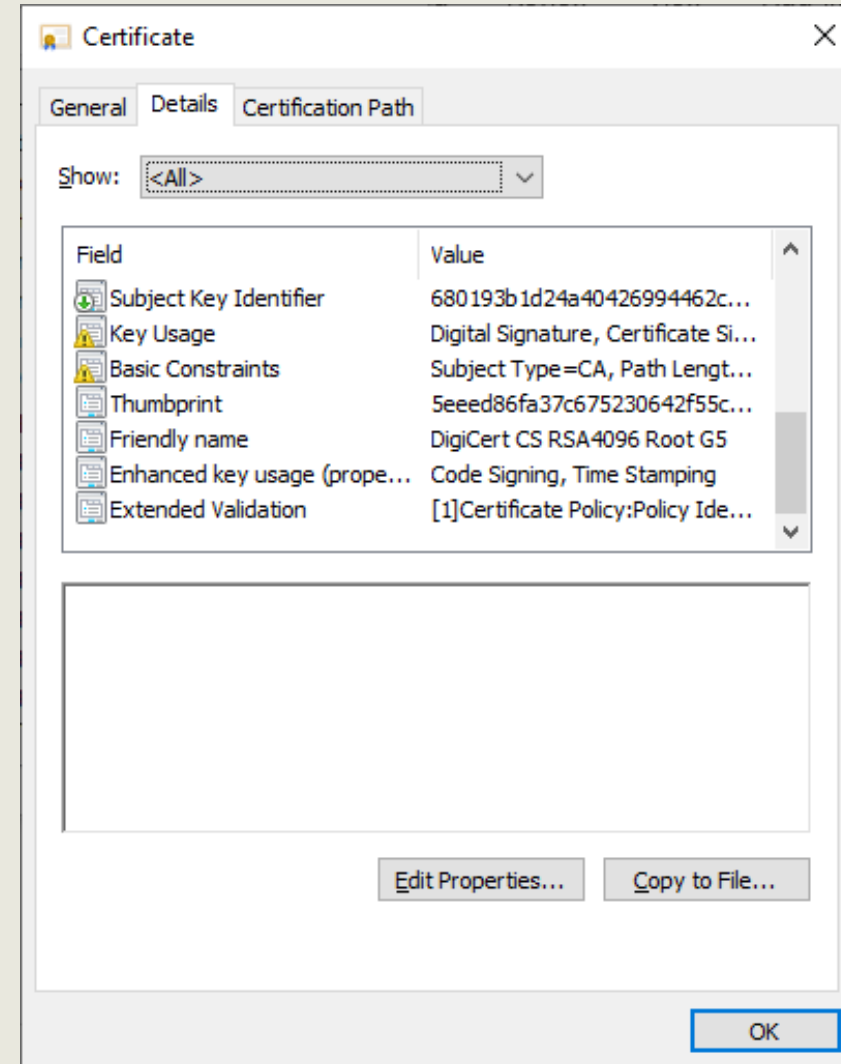
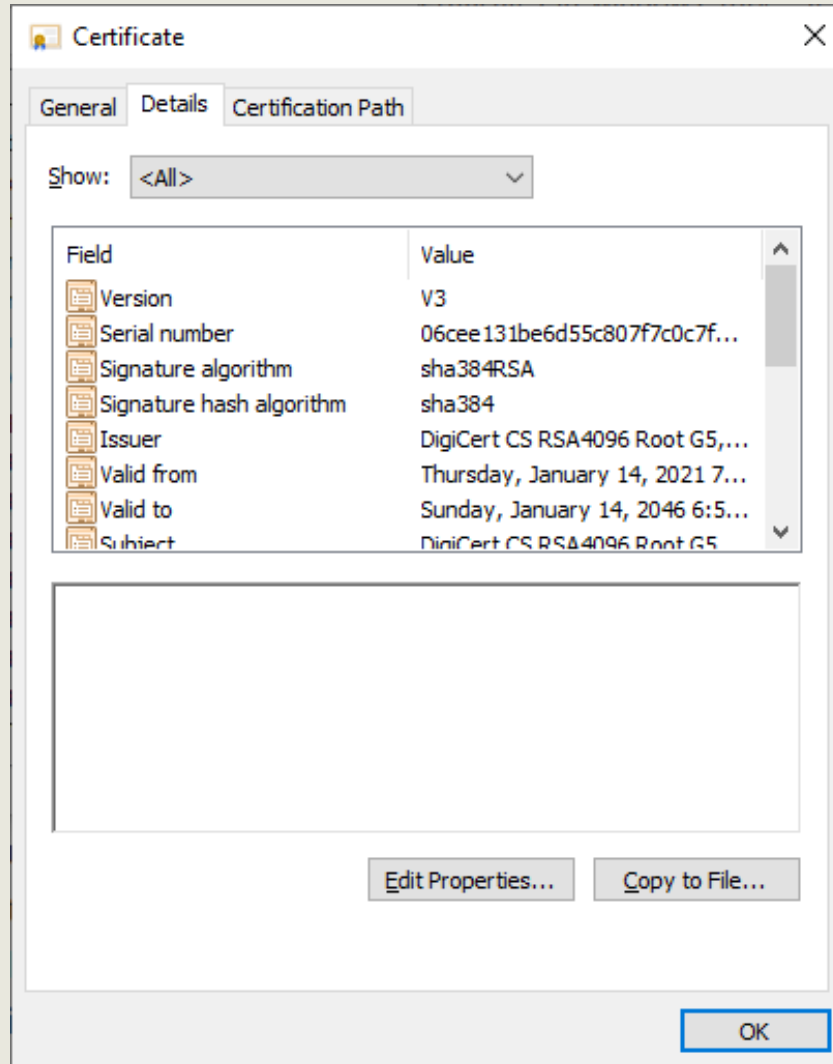


DIGITAL CERTIFICATES

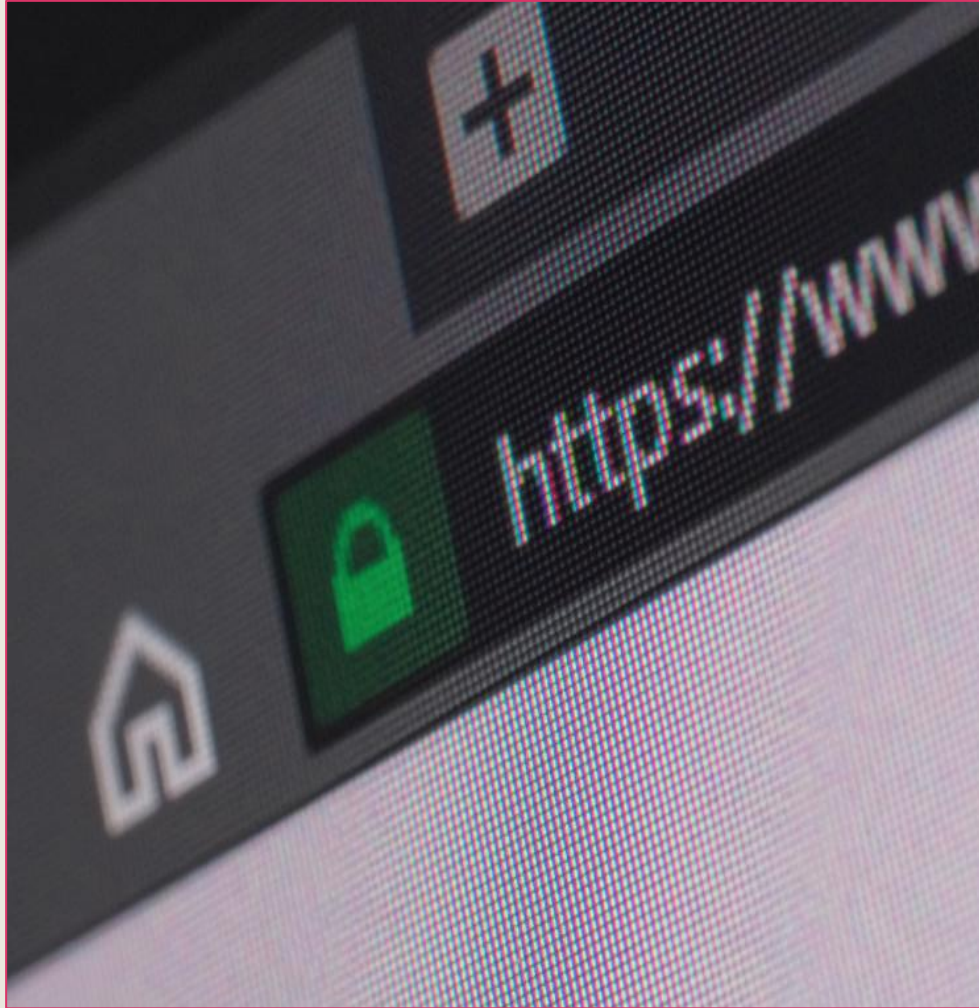
- A digital certificate is a form of file used to map cryptographic key pairs to entities such as individuals, web sites, or organizations
- If public trust is needed, then a trusted certificate authority (CA) will assume the role of a third party to validate, identify, and associate them with cryptographic pairs using the digital certificates
- The owner of the private key can then use it to sign documents, while the public key is used to verify the validity of those signatures



DIGITAL CERTIFICATES



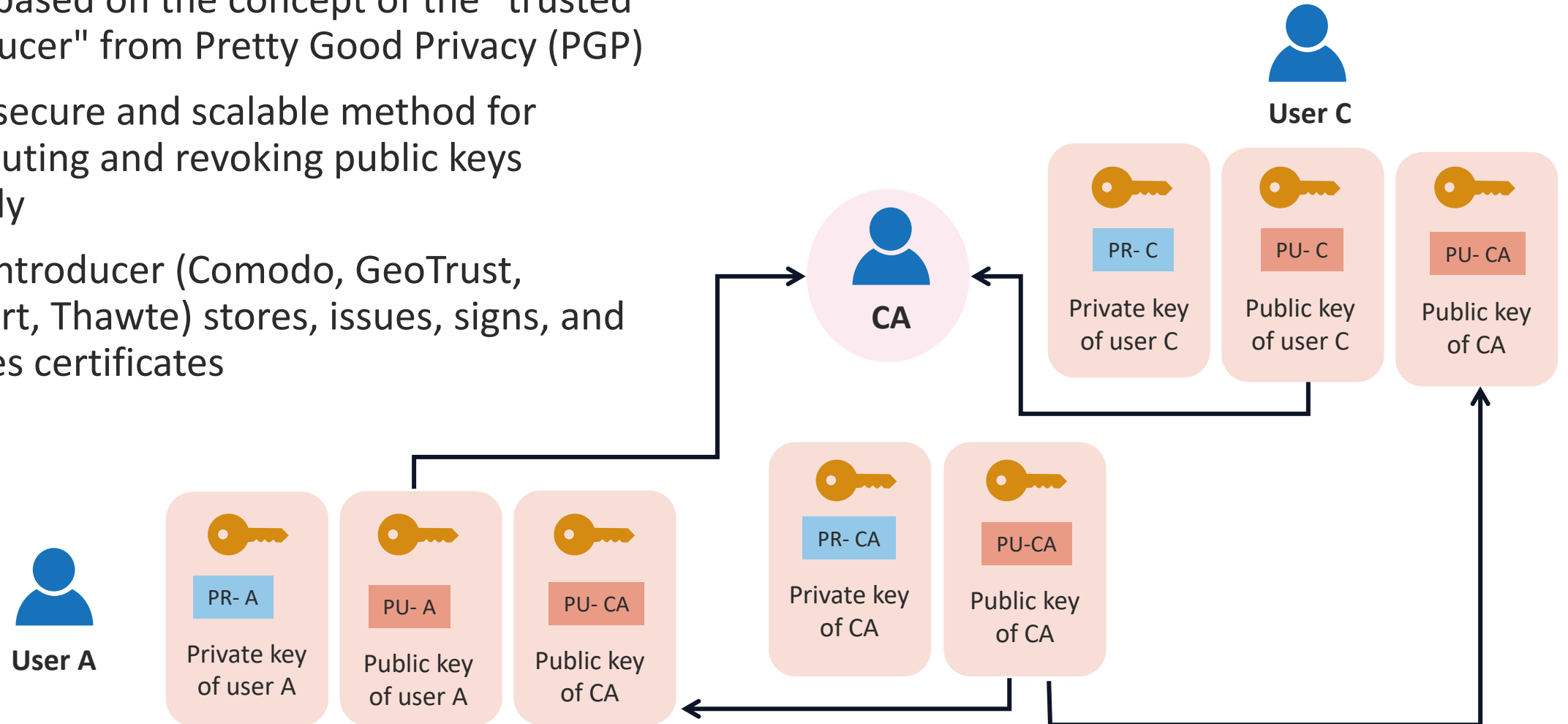
DIGITAL CERTIFICATES



- The most common commercial format for digital certificates is based on the X.509 standard, which consists of the following:
 - A public key
 - A pseudorandom hexadecimal serial number linked to the certificate
 - Information about the issuing CA
 - A lifetime (usually 1 to 5 years)
 - A digital signature
 - The hashing and signing algorithms used
 - Hashing – SHA1, SHA2, SHA384
 - Signing – DSA, RSA, and ECDSA
 - Extensions for more features like security

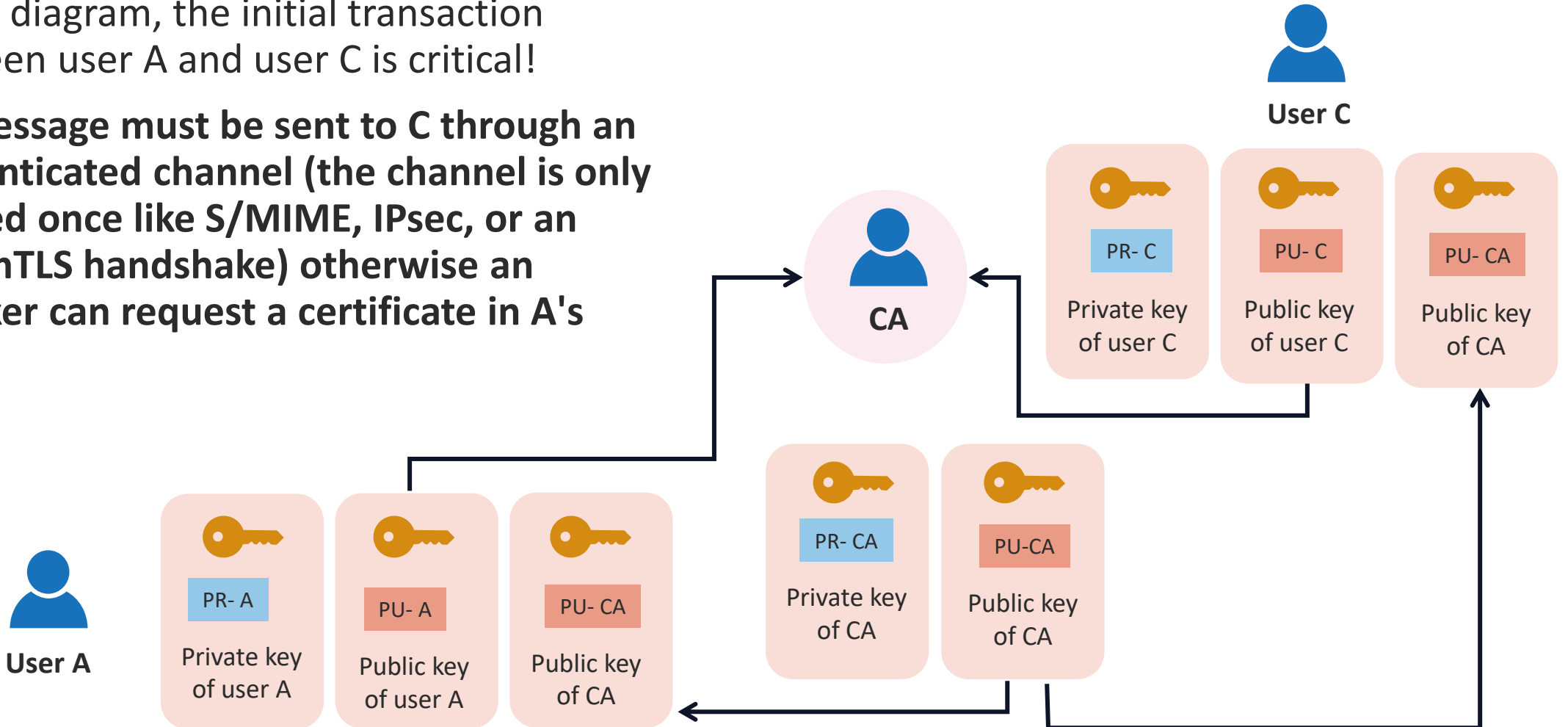
PUBLIC KEY INFRASTRUCTURE

- PKI is based on the concept of the "trusted introducer" from Pretty Good Privacy (PGP)
- It is a secure and scalable method for distributing and revoking public keys globally
- A CA introducer (Comodo, GeoTrust, DigiCert, Thawte) stores, issues, signs, and revokes certificates



PUBLIC KEY INFRASTRUCTURE (PKI)

- In this diagram, the initial transaction between user A and user C is critical!
- A's message must be sent to C through an authenticated channel (the channel is only needed once like S/MIME, IPsec, or an SPA/mTLS handshake) otherwise an attacker can request a certificate in A's name



PUBLIC KEY INFRASTRUCTURE

- Note that with X509.v3 certificates – if you look at the fields – 2 public key algorithms are involved
- The one within the certificate, the Subjects Public Key algorithm is often some 160-bit elliptic curve mechanism
- AND the one that was used to SIGN the certificate - like RSA 2048 or ECDSA



WHY REVOKE A CERTIFICATE?

- A server's private key is leaked so its TLS certificate must be revoked immediately
- A user's device is lost or stolen, risking exposure of client certificates
- An AuthN certificate is revoked when the subject is a mover or leaver
- Role changes are affecting access privileges
- Service decommissioning to revoke all associated certificates
- Migration to a new domain or infrastructure where old certificates become obsolete
- CA issues a certificate with incorrect subject information or usage constraints



PKI CERTIFICATE REVOCATION

- The **certificate revocation list (CRL)** is the original method using a list of certificates that are invalid serial numbers and it is:
 - Issued by the CA who granted the certificate
 - Generated and published periodically
 - Pushed or polled in defined intervals or immediately – not real-time
 - Downloaded by client regularly but also not in real-time
- A CA can perform suspension of a certificate instead and that DOES NOT place the serial number on the CRL

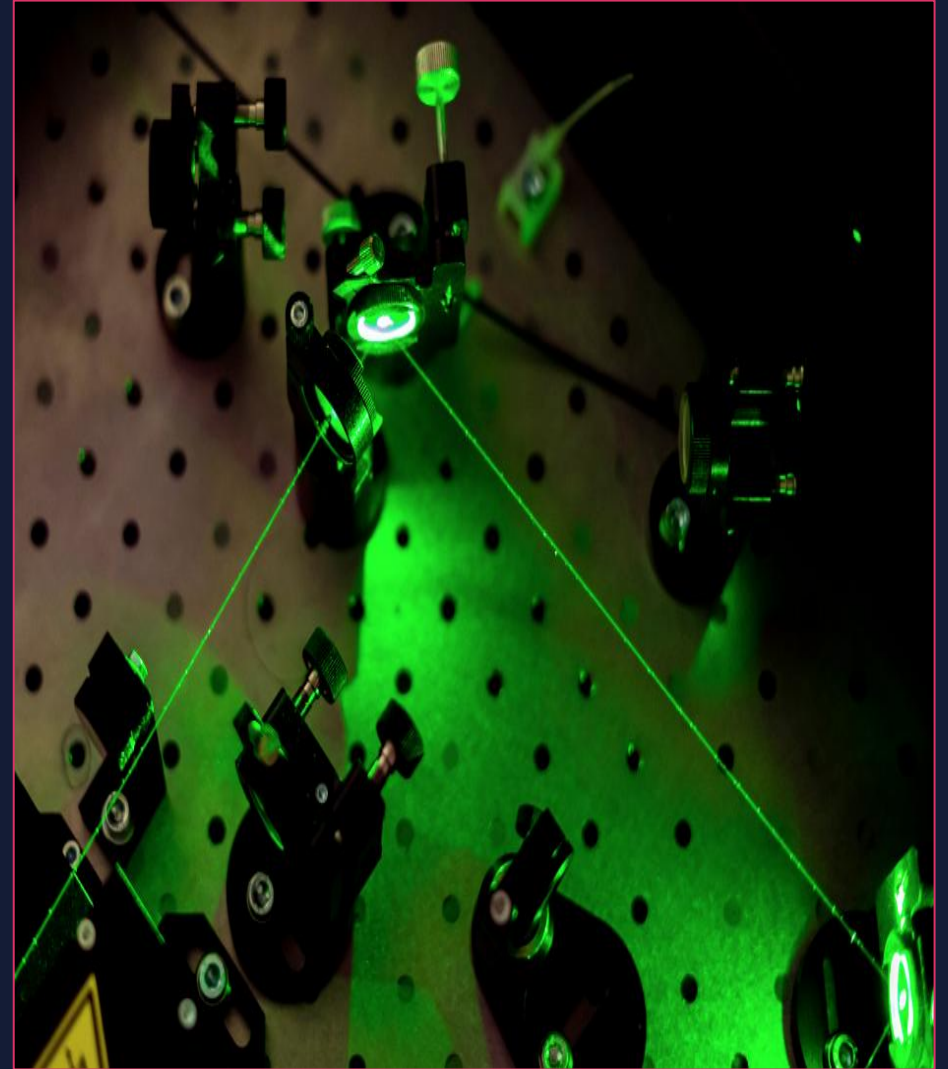


PKI CERTIFICATE REVOCATION

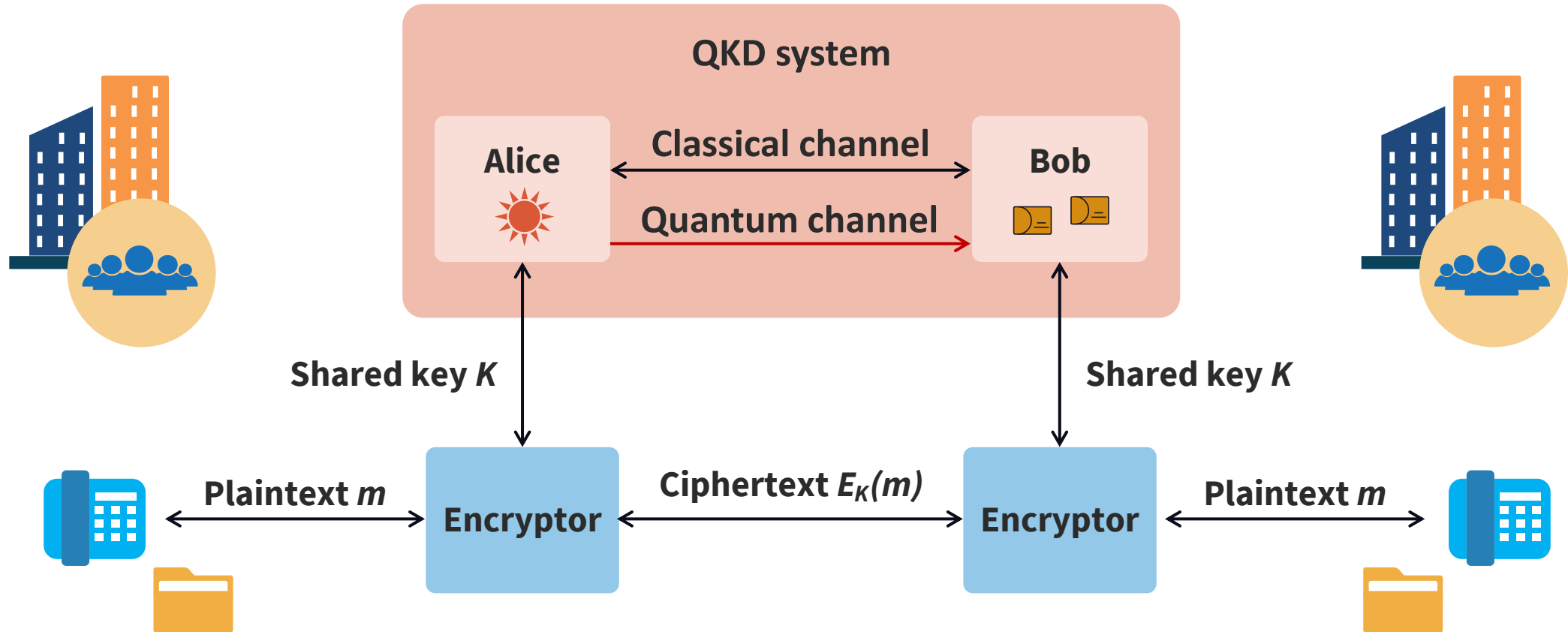
- **Online Certificate Status Protocol (OCSP)** is a database evolution of the CRL
- It is a method for a web browser or other client to determine the validity of a SSL/TLS certificate by verifying with the CA vendor
- It is a dynamic online transactional database of serial numbers that is generated and published immediately
- Clients query database anytime (although many vendors will bypass)
 - OCSP improves security, but causes websites to load slower

QUANTUM KEY DISTRIBUTION (QKD)

- QKD is a secure communication scheme that uses a cryptographic protocol based on quantum mechanics
- It allows two entities to generate a shared random secret key which can then be used to encrypt and decrypt
- The process of QKD is **not** the same as the quantum cryptography
- It is considered a possible "post-quantum" solution to implement beyond elliptic curve Diffie-Hellman ephemeral (ECDHE)



QUANTUM KEY DISTRIBUTION



Brute Force Attacks

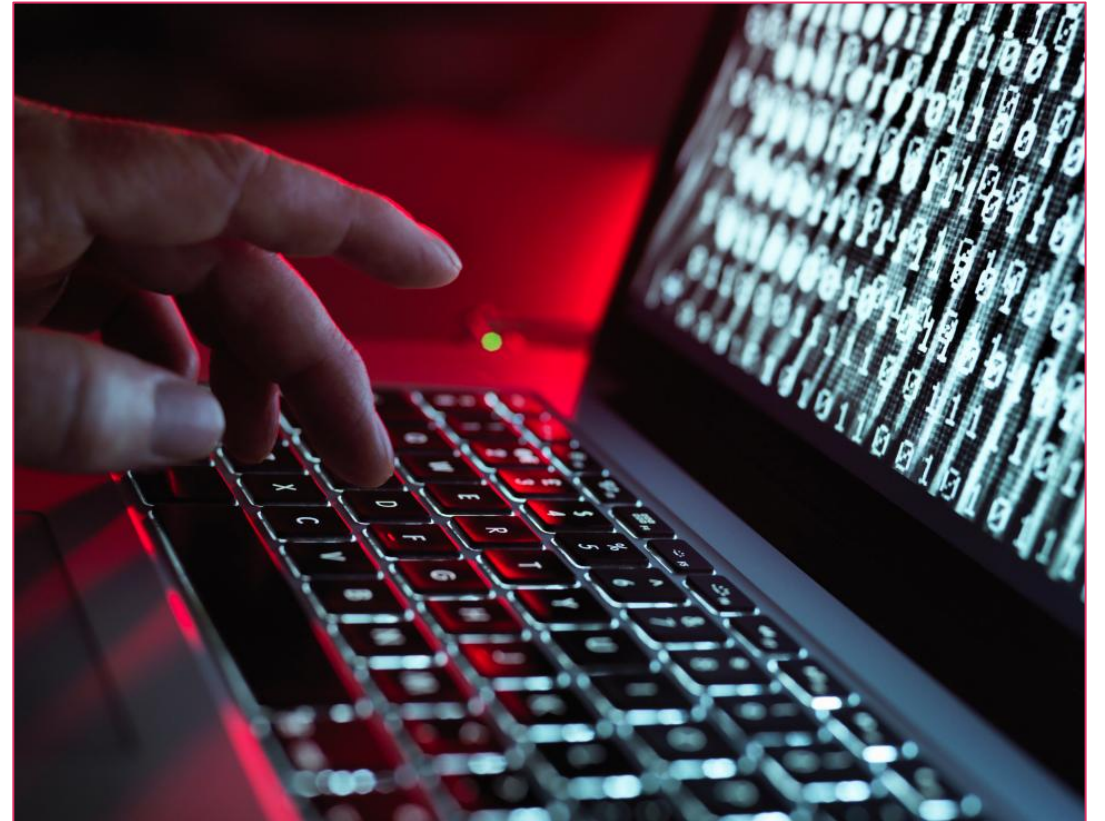
- Brute Force Attacks
 - **Definition:** The attacker tries every possible key until the correct one is found
 - Use Case: Most effective against weak encryption algorithms or short key lengths
 - Mitigation: Use strong algorithms with long key sizes (e.g., AES-256), implement rate limiting, and monitor for abnormal access patterns
- Ciphertext-Only Attack
 - **Definition:** The attacker has access only to encrypted data (ciphertext), with no knowledge of the plaintext or key
 - Strategy: Analyze patterns or statistical anomalies in the ciphertext to infer the plaintext or key
 - Mitigation: Ensure ciphertext appears random and use strong encryption schemes that resist statistical analysis

Brute Force Attacks

- Known Plaintext Attack
 - **Definition:** The attacker possesses both the plaintext and its corresponding ciphertext
 - Use Case: Common in intercepted communications where some content is predictable (e.g., headers, greetings)
 - Goal: Use known pairs to deduce the encryption key or algorithm behavior
 - Mitigation: Employ algorithms with strong diffusion and confusion properties to obscure relationships
- Chosen Ciphertext Attack
 - **Definition:** The attacker can submit ciphertexts to a decryption oracle and observe the resulting plaintexts
 - Goal: Use the decrypted outputs to reverse-engineer the key or exploit weaknesses in the algorithm
 - Mitigation: Implement secure padding schemes (e.g., RSA-OAEP), restrict decryption access, and monitor for abuse

IMPLEMENTATION ATTACKS

- Implementation attacks will commonly include the following:
 - Taking advantage of weak initialization vectors and nonces in various algorithms
 - Attacking weaknesses in key management such as lack of rotation and unsecure backups
 - Exploiting vulnerabilities in the underlying deployment such as the Dragonblood attack on WPA3
 - A focus on exploiting weaknesses in the software code itself





SIDE-CHANNEL ATTACKS

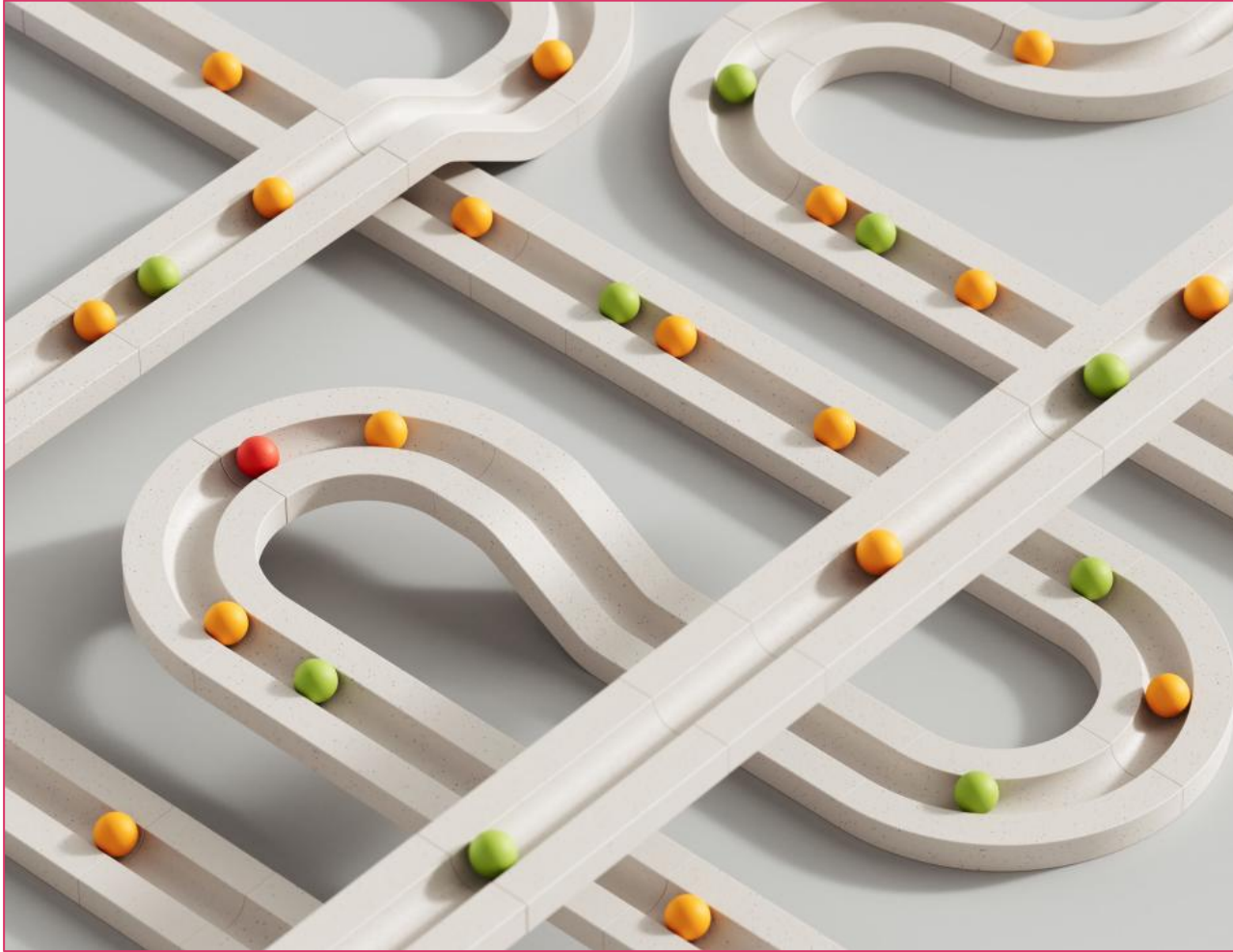
- A side-channel attack (SCA) is a passive attack where the attacker can collect information based on power consumption, electromagnetic radiation, execution time, and other attributes
- The immediate value of each of these sources depends on the data being processed in the module (such as a smart card), which in turn depends on the secret key
- SCA can also be launched by combining information from many leakage traces collected at different inputs
- This is called differential power analysis (DPA) – an even more serious threat

ON-PATH ATTACKS

- An on-path attack (**previously called man-in-the middle - MITM**) is when an adversary puts themselves between two devices (such as a web browser and a server) and wiretaps or changes communications between the two hosts
- The attackers can then gather information as well as spoof or masquerade as either of the two hosts

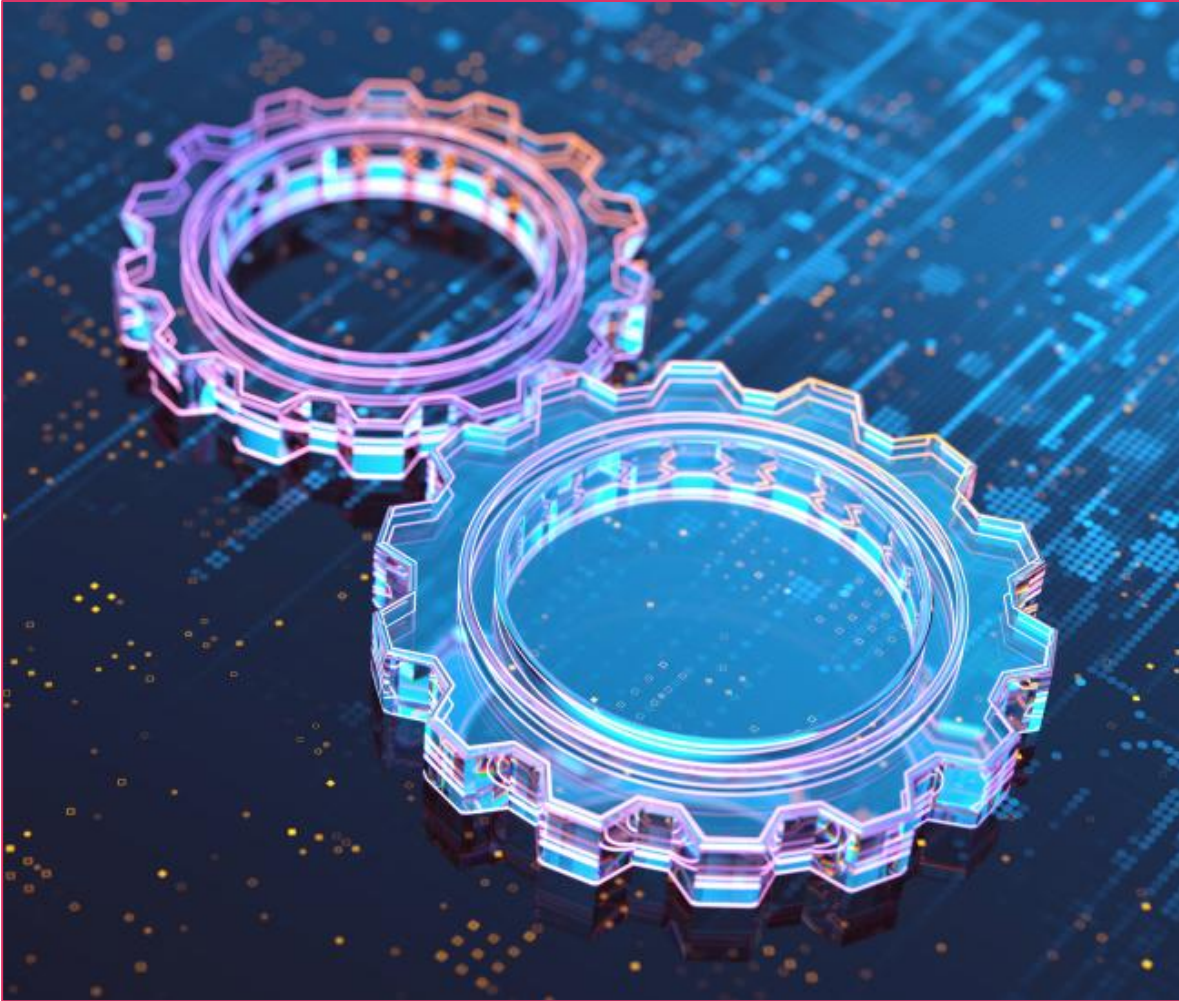


ON-PATH ATTACKS



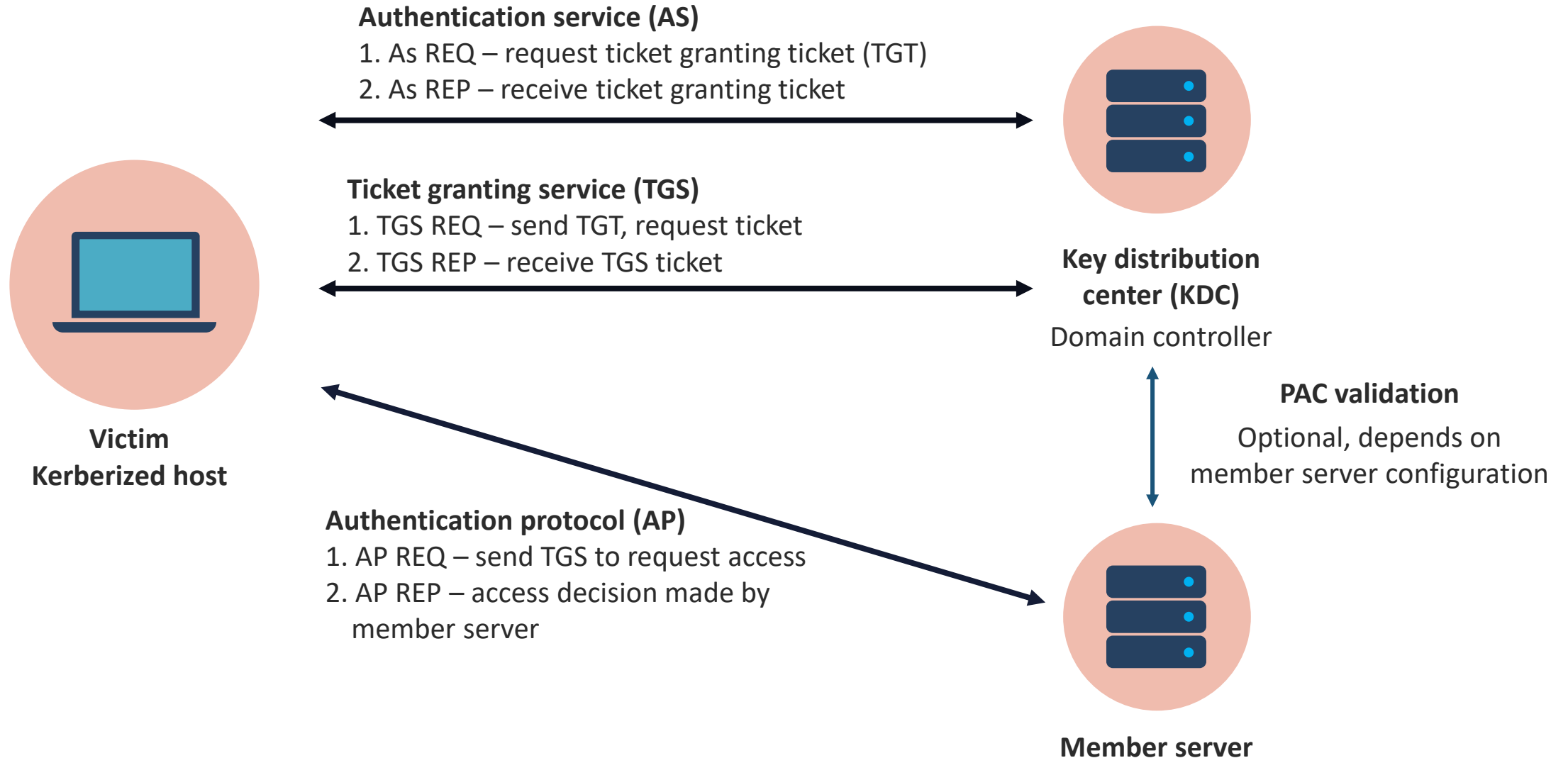
- Besides web applications, on-path attacks can target ARP and ICMP messages, email communications, and public/hotel Wi-Fi networks
- DNS servers are also common as victims suffer DNS spoofing and hijacking, sending users to a rogue site that gathers sensitive data
- The usual targets of on-path attackers are SaaS providers, e-commerce businesses, and financial app users

EXPLOITING KERBEROS



- **Kerberoasting**
 - Where an attacker removes encrypted Kerberos tickets from the network and tries to crack the encryption and password hashes for service accounts in Active Directory
 - If successful, they can get access to sensitive data or network resources

KERBEROASTING ATTACK



EXPLOITING KERBEROS

- **Pass-the-ticket attacks**
 - A lateral movement exploit where attackers steal a Kerberos ticket from one system and use it to get access to another system by reusing the stolen ticket
- **Elevation of privilege vulnerabilities discovered by Google Project Zero**
- **Downgrade of encryption from AES to MD4-RC4**
- Group Policy, change, configuration, and patch management are the keys to mitigation

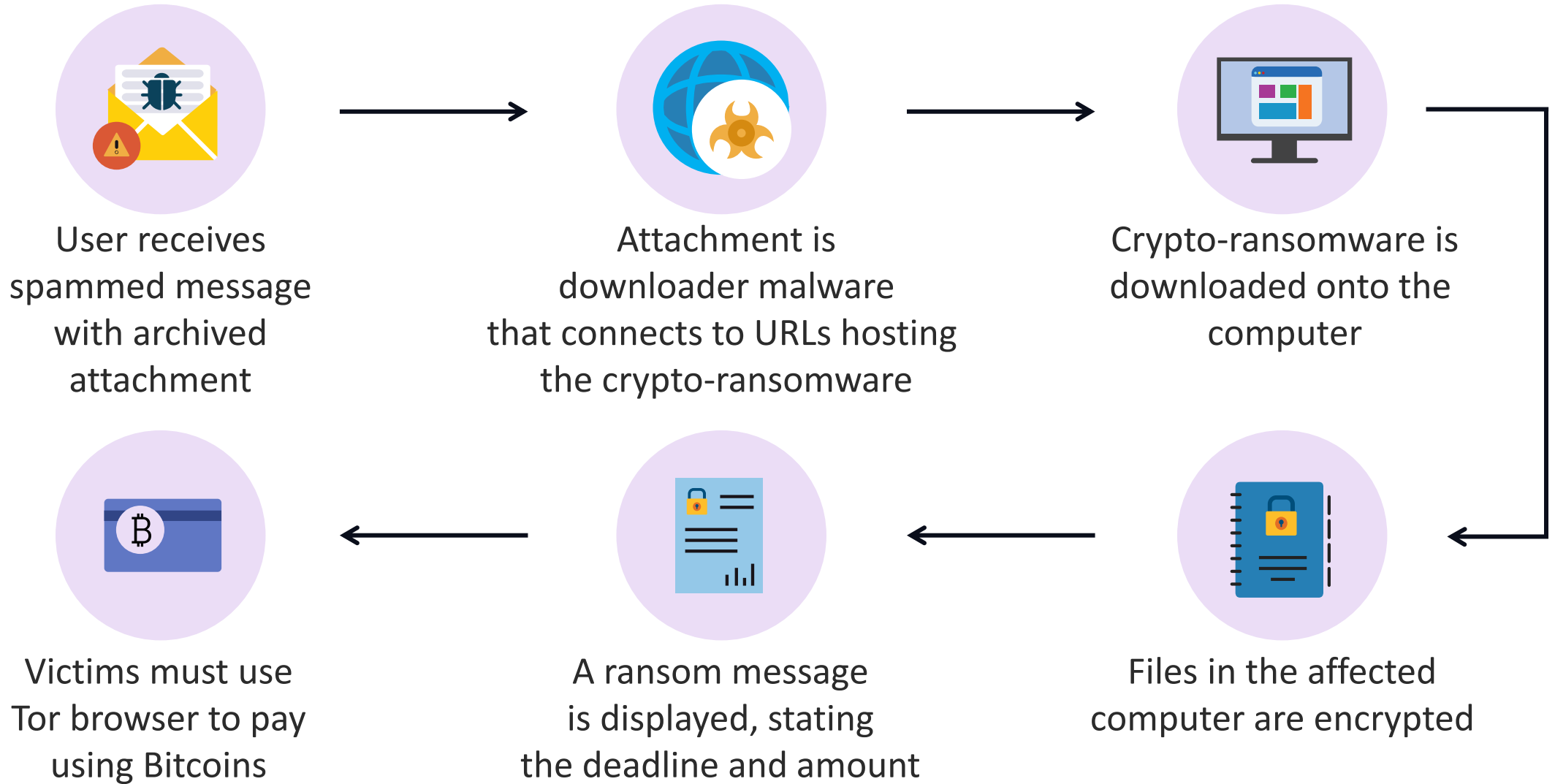


EXPLOITING KERBEROS



- **Golden Ticket attacks**
 - Where a threat agent handles the Kerberos authentication protocol to get unrestricted access to an organization's entire realm or domain (including devices, files, and domain controllers)
 - This hack lets unauthorized users exploit the krbtgt account, a key aspect of the Kerberos system responsible for encrypting and signing all tickets

RANSOMWARE LIFE CYCLE





DEFENDING AGAINST RANSOMWARE ATTACKS

- Maintain a precise and timely inventory of all assets
- Deploy two top-of-the-line antivirus programs
 - Make sure that antivirus is configured to automatically scan emails
- Apply tested patches and updates to all endpoints
- Consider managed security services (MSSP) that block access to known ransomware sites and zero days
- Backup and restore all data to an external site or cloud service
- Conduct security and awareness and enforce AUPs